A PROJECT REPORT

on

“Learning Cache Replacement from Belady: DAgger Training from Single Applications to Shared Caches”

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR’S DEGREE IN

COMPUTER SCIENCE & ENGINEERING

BY

Shivansh Batra	**23051784**
Priyam Kumar	**23051690**
Shristi Mishra	**23051788**
Jatin Aryan	**23053366**
Shreya Singh	**23051954**

UNDER THE GUIDANCE OF

Dr. Bindu Agarwalla

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

November 2026

KIIT Deemed to be University
School of Computer Science and Engineering
Bhubaneswar, ODISHA 751024

CERTIFICATE

This is certify that the project entitled

“Learning Cache Replacement from Belady: DAgger Training from Single Applications to Shared Caches”

submitted by

Shivansh Batra	**23051784**
Priyam Kumar	**23051690**
Shristi Mishra	**23051788**
Jatin Aryan	**23053366**
Shreya Singh	**23051954**

is a record of bonafide work carried out by them, in the partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science & Engineering) at KIIT Deemed to be University, Bhubaneswar. This work is done during year 2026-2027, under our guidance.

Date: 11/11/2026

(Dr. Bindu Agarwalla)
Project Guide

Acknowledgements

We are profoundly grateful to Dr. Bindu Agarwalla of the School of Computer Science and Engineering, KIIT Deemed to be University, for the expert guidance and continuous encouragement from the commencement of this project to its completion.

We also thank the School of Computer Science and Engineering for the laboratory and computing facilities, and our families and friends for their support. The simulations, the model training and the web simulation of this project were run on a personal laptop, so the long experiments needed patience from everyone around us.

Signature of Shivansh Batra

Signature of Priyam Kumar

Signature of Shristi Mishra

Signature of Jatin Aryan

Signature of Shreya Singh

ABSTRACT

A cache that is full must decide which line to evict. Belady's optimal policy evicts the line that will be used farthest in the future. It gives the lowest possible miss count but needs the future, so real last-level caches (LLC) use heuristics such as LRU and RRIP. This project asks three questions. Can small machine-learning models that imitate Belady, trained with DAgger, close a large part of the gap between LRU and Belady? Should every application carry its own model, or is one generic model enough? And what happens when several applications share the cache?

We built the whole pipeline ourselves: four real C programs (hash table, graph traversal, matrix multiplication, convolution) traced with Valgrind; an L1/L2/LLC simulator in C++ with eight baseline policies and Belady; a labeler that records the Belady victim at every eviction; three model families (MLP, LightGBM trees, LSTM with attention); DAgger training; experiments on held-out test inputs that were used once; and a static web simulation. The MLP closes 36% to 47% of the LRU-to-Belady gap on hash and graph (best simple policy 23% to 35%) and 100% on matrix and conv. A per-application model does not beat one generic model of the same size. Models trained on a single application fail on other applications (down to −425%) and in a shared cache. DAgger on the merged stream of the four applications fixes the shared case: on a held-out mixture the best models have 4% to 9% fewer misses than the best simple policy at 2 MB and up to 16% fewer at 8 MB. An ownership rule stays behind simple policies. Software decision times (7 to 146 µs) are far above a hardware budget of about 0.1 µs, so the results show potential in simulation, not a deployable design.

Keywords: cache replacement, last-level cache, Belady's algorithm, imitation learning, DAgger, LSTM, LightGBM, shared cache, simulation.

Contents

Sl. No. Chapter / Section **Page No.**

1 Introduction....1

2 Basic Concepts / Literature Review....3

2.1 Caches and the last-level cache....3
2.2 Replacement policies used as baselines....3
2.3 Belady's optimal policy and the gap closed....3
2.4 Learned cache replacement: related work....4
2.5 Imitation learning and DAgger....4
2.6 Model families....4
2.7 Several applications sharing a cache....5

3 Problem Statement / Requirement Specifications....6

3.1 Project Planning....6
3.1.1 Research questions and objectives....6
3.1.2 Work plan....6
3.2 Project Analysis (Requirement Specification)....7
3.2.1 Functional requirements....7
3.2.2 Non-functional requirements....7
3.2.3 Risks and how we handled them....7
3.3 System Design....8
3.3.1 Design Constraints....8
3.3.2 System Architecture / Block Diagram....8

4 Implementation....11

4.1 Methodology / Proposal....11
4.1.1 Workloads and inputs....11
4.1.2 Tracing and the L1/L2 filter....11
4.1.3 Belady labeler and dataset....11
4.1.4 Models....12
4.1.5 DAgger variants and what drives each data set....12
4.1.6 Experiments....13
4.2 Testing / Verification Plan....13
4.3 Result Analysis / Screenshots....14
4.3.1 E1 and E2: learned policies against baselines....14
4.3.2 E3: specialized or generic?....15
4.3.3 E4: models do not transfer between applications....16
4.3.4 E5: other LLC configurations without retraining....16
4.3.5 E6: cost of storing and running the models....17
4.3.6 E7: switching between applications....19
4.3.7 E8: four applications in one shared LLC....20
4.3.8 Extension 1: DAgger for the LSTM....21
4.3.9 Extension 2: mixed-stream DAgger for the shared cache....21
4.3.10 Static web simulation....24
4.4 Quality Assurance....30

5 Standards Adopted....32

5.1 Design Standards....32

5.2 Coding Standards....32
5.3 Testing Standards....32
6 Conclusion and Future Scope....34
6.1 Conclusion....34
6.2 Future Scope....34
References....36
Individual Contribution....37
Plagiarism Report....42

List of Figures

Figure 1.1: The project pipeline, from real programs to results. The model sees only the past; the future exists only in the labeler.....2
Figure 3.1: DAgger rounds. In single-stream DAgger the cache holds one application; in mixed-stream DAgger the cache is shared by four.....9
Figure 3.2: The three arbitration rules for one set (illustration; colours are owning applications and the requester is the orange application).....10
Figure 4.1: Gap closed on the test inputs, learned models against the best simple policy.....15
Figure 4.2: Specialized and generic MLPs on the hash and graph test inputs.....16
Figure 4.3: Cross-application results (E4).....16
Figure 4.4: Gap closed under other LLC configurations.....17
Figure 4.5: Model size against software decision time (E6).....19
Figure 4.6: Switching: effect of the slice length (E7).....20
Figure 4.7: Shared LLC results (E8).....21
Figure 4.8: Extension 2: results on the test mixture, 2 MB LLC.....22
Figure 4.9: Extension 2: results on the test mixture, 8 MB LLC.....23
Figure 4.10: Extension 2: model families under the generic rule.....24
Figure 4.11: Web simulation: Overview screen (full-run results).....25
Figure 4.12: Web simulation: Single application screen with an animated sample window (hash, MLP specialized against LRU and Belady) and the full-run table.....26
Figure 4.13: Web simulation: Switching screen (E7).....27
Figure 4.14: Web simulation: Shared cache screen, validation and held-out test results of the mixed-stream extension.....28
Figure 4.15: Web simulation: sample window of the shared cache (4,000 accesses), colours show the owning application.....29
Figure 4.16: Web simulation: Decision inspector, one eviction with the 16 candidate lines, the model scores and the Belady and LRU victims.....29
Figure 4.17: Web simulation: Results and tables screen.....30

List of Tables

Table 1: Related work and how this project relates to it....4
Table 2: Work plan and status of the project stages....6
Table 3: Main risks and the measures taken....7
Table 4: Cache configuration used in all experiments....8
Table 5: Components of the system....8
Table 6: The four workloads....11
Table 7: Inputs per split (full arguments in configs/inputs.tsv)....11
Table 8: Model families as built....12
Table 9: Data sets and models by the policy that drove the cache....12
Table 10: Experiments....13
Table 11: Test cases....13
Table 12: E2: mean gap closed on the two test inputs of each application....14
Table 13: E3: gap closed (mean ± standard deviation over three seeds), MLP....15
Table 14: E4: gap closed, model of one application (rows) run on another (columns)....16
Table 15: E5: gap closed in other LLC configurations (selected rows)....17
Table 16: E6: model cost....18
Table 17: E6: storage of four specialized models against one generic model....18
Table 18: E6: activation time per switch (µs) for the mean model size of each family....18
Table 19: E7: LLC misses against LRU (positive = fewer misses), by slice length....19
Table 20: E8: shared 2 MB LLC, four applications....20
Table 21: Extension 1: mean gap closed, LSTM DAgger....21
Table 22: Extension 2, test mixture, 2 MB: gap closed / change in misses against the best simple policy (DRRIP)....21
Table 23: Extension 2, test mixture, 8 MB: gap closed / change in misses against the best simple policy (BIP)....22

Chapter 1
Introduction

Modern processors hide the slowness of main memory behind a hierarchy of caches. The last-level cache (LLC) is the final stop before memory: a miss there costs on the order of a hundred nanoseconds, hundreds of cycles. Which line to throw out when a set is full is therefore one of the most studied decisions in computer architecture. The best possible choice is known: Belady's algorithm evicts the line whose next use lies farthest in the future [1]. It cannot be built in hardware, because it needs the future, but it can be computed offline on a recorded trace. Practical policies such as LRU, RRIP [2] and bimodal insertion [3] are cheap approximations, and on many programs a large gap to Belady remains.

Recent research uses machine learning to close this gap. Hawkeye [4] and Glider [5] learn from Belady's decisions on past accesses, and PARROT [6] frames replacement as imitation learning of Belady with a neural network. These papers study one program or one trace at a time. This project asks a different set of questions that become important when the cache is shared by several programs, as it is on every multicore processor and on every time-sliced one. Should each application **carry its own small model** for its own replacement decisions, which an operating system or hardware layer activates when the application runs? Or is one generic model, trained on all applications, just as good and much cheaper to store? What do such models cost, in storage, in decision time and when they are switched? And what happens to models trained on one application alone when four applications share the same LLC?

To answer these questions with real evidence, we built a complete experimental pipeline (Figure 1.1). Four real C programs with very different memory behavior are traced under Valgrind. A C++ simulator filters the traces through simulated L1 and L2 caches and replays the resulting LLC accesses under eight baseline policies. A labeler replays the same trace with Belady and records, at every eviction, which lines Belady would have evicted. This produces the training data. Three model families (a small MLP, LightGBM trees and an LSTM with attention) are trained to imitate Belady, and DAgger is used to correct the mismatch between the states seen in training and the states the learned policy creates itself. Training, model selection and testing use different program inputs, and the test inputs were used once.

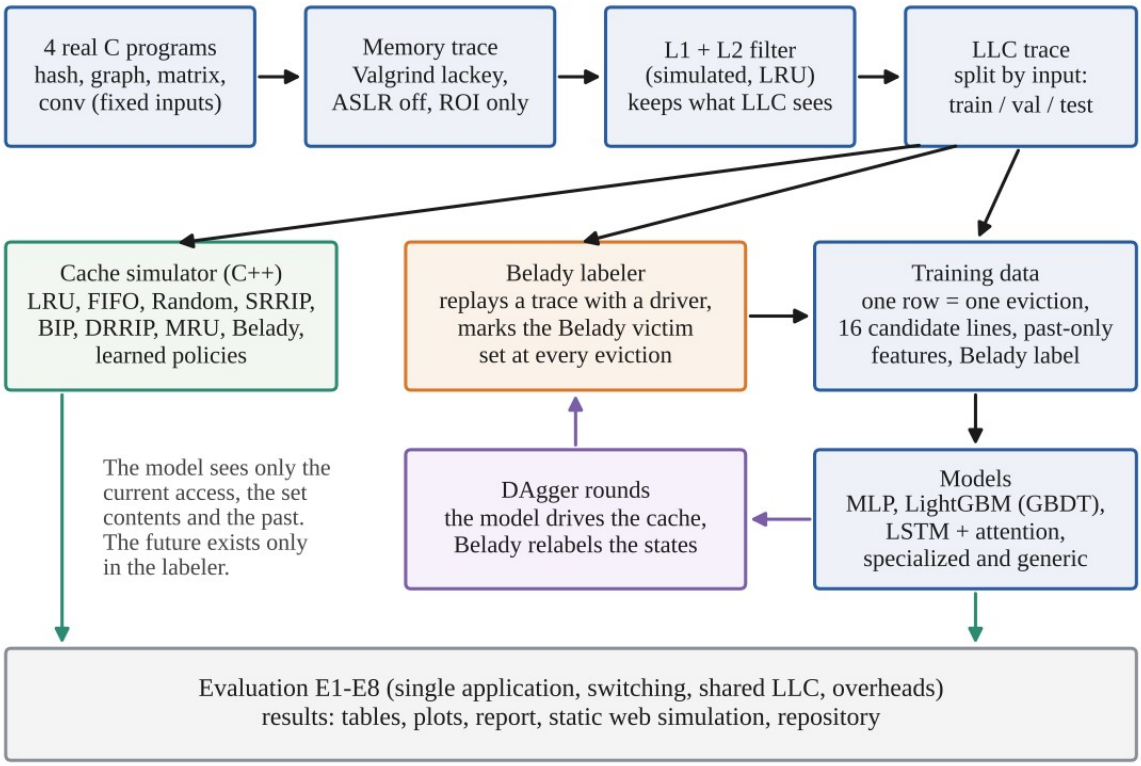


Figure 1.1: The project pipeline, from real programs to results. The model sees only the past; the future exists only in the labeler.

Main findings. (1) A learned policy beats the best simple policy on held-out inputs of all four applications: the MLP closes 46.3% to 47.1% of the LRU-to-Belady gap on hash, 35.8% to 36.2% on graph and 100% on matrix and conv (Table 12). (2) A model trained for one application is not better than one generic model of the same size; it is worse than useless on other applications (Table 14), probably because a rule that suits one memory pattern is harmful on another. (3) Software inference costs 7 to 146 µs per decision and 7 KB to 1 MB per model, far above a hardware budget of roughly 100 ns (Chapter 4). (4) Models trained on one application at a time lose to DRRIP and BIP when four applications share the LLC. Training DAgger on the merged stream of all four applications fixes this: on a held-out mixture the generic models have 5% to 9% fewer misses than the best simple policy at 2 MB (Section 4.3.9). (5) An ownership rule that lets each model evict only its own lines never beat the best simple policy.

Organization of the report. Chapter 2 gives the concepts and the related work. Chapter 3 states the problem, the requirements and the system design. Chapter 4 describes the implementation, the tests, all results (experiments E1 to E8 and two extensions) and the web simulation. Chapter 5 lists the standards followed, Chapter 6 concludes and gives the future scope. The references, the individual contribution reports and the plagiarism report follow.

Chapter 2
Basic Concepts / Literature Review

This chapter explains the ideas a reader needs to follow the project: how a set-associative cache works, which replacement policies we compare with, what Belady's algorithm and the "gap closed" metric are, how learned replacement and imitation learning work, and what is known from earlier research.

2.1 Caches and the last-level cache

A cache stores copies of recently used memory lines of 64 bytes. A **set-associative** cache of size S with W ways and line size 64 B has S / (64 × W) sets. An address maps to exactly one set; inside the set it may occupy any of the W ways. On an access the cache looks for the line in the set: a **hit** returns the data quickly, a **miss** fetches the line from the next level and, if all W ways are valid, must **evict** one of them. The decision which one is the replacement policy. Our LLC has 2 MB and 16 ways, which gives 2,048 sets and 32,768 lines (Table 4).

The LLC is shared. On a multicore processor all cores use it; on a single core it is shared in time by all programs the operating system schedules. Lines of different programs compete for the same sets, and one program's decisions change the miss rate of the others. This sharing is what makes the per-application question interesting. We measure cache quality by the **miss count** and by **MPKI** (misses per thousand instructions). Because we have no timing model, a policy cannot change the number of instructions of a trace, so MPKI ratios equal miss ratios.

2.2 Replacement policies used as baselines

- **LRU** evicts the least recently used line. It is optimal for programs with strong temporal locality and the worst case for cyclic sweeps larger than the cache.
- **FIFO** evicts the oldest line and **Random** a random line. They are cheap reference points.
- **MRU** evicts the most recently used line. It is the right rule for cyclic sweeps. We added it after seeing that the matrix and conv workloads behave like sweeps, so a learned policy has to beat it and not only LRU.
- **SRRIP** [2] keeps a 2-bit re-reference prediction value per line. New lines enter with a "long" prediction, hits promote lines, and the line with the largest value is evicted. This resists scans.
- **BIP** [3] inserts most new lines at the LRU position and only one in 32 at the MRU position, so a thrashing working set is partly kept. **BRRIP** is the same idea for RRIP. **DRRIP** [2] chooses between SRRIP and BRRIP by set dueling: a few leader sets run each policy and a 10-bit counter picks the better one for all other sets.

In every experiment we also report the **best simple policy**: for each workload or mixture the best of LRU, SRRIP, DRRIP, BIP and MRU measured on the same trace. A learned policy that does not beat it has no practical value.

2.3 Belady's optimal policy and the gap closed

When a line must be evicted, Belady's algorithm (MIN) evicts the resident line whose next access is farthest in the future, or that is never used again [1]. For a cache that must fetch every missed line (no bypass), as ours does, this minimizes the number of misses. We check this in our tests against an exhaustive search on tiny traces (Section 4.2). Belady needs the future, so it is only an offline teacher and an upper bound.

To compare policies across traces of very different difficulty we use the **gap closed**: for a policy with M misses on a trace where LRU has M_x and Belady M_β, gap closed = $(M_x - M) / (M_x - M_\beta)$. It is 0% for LRU, 100% for Belady and negative for policies worse than LRU. On traces where LRU already equals Belady there is nothing to gain; such rows are not counted.

2.4 Learned cache replacement: related work

Table 1 lists the work that this project builds on. All of it is cited from the original papers; we did not run the original code of any of them, so we make no numeric comparison with their published results.

Table 1: Related work and how this project relates to it

Work	Idea	Relation to this project
Belady 1966 [1]	Evict the line used farthest in the future.	Teacher for the labels and upper bound for all results.
RRIP, DRRIP 2010 [2]	Re-reference interval prediction; set dueling between SRRIP and BRRIP.	Baselines (SRRIP, BRRIP, DRRIP).
DIP, BIP 2007 [3]	Adaptive insertion policies; bimodal insertion.	Baseline (BIP).
Hawkeye 2016 [4]	Reconstructs Belady's decisions on past accesses and trains a PC-based predictor of cache-friendly and cache-averse loads.	Same teacher; we imitate the victim choice directly.
Glider 2019 [5]	Offline attention-based LSTM guides the design of an online predictor over the history of PCs.	Our LSTM with attention follows the same idea in a much smaller form.
PARROT 2020 [6]	Replacement as imitation learning of Belady with an attention-based neural network.	Our LSTM is a small PARROT-inspired model; differences are listed in Section 4.1.4.
Mockingjay 2022 [7]	Predicts reuse distances to mimic Belady's MIN more closely.	Not implemented; named as future work.

What is different here is the question rather than the model. The papers above study one program or one trace set. We ask whether an application should **carry** its own model, compare it with a generic model of equal size and of equal total storage, measure the cost of storing and activating the models, and study what happens when four applications share one LLC, including how to train for that case. We have not done an exhaustive literature search, so we make no claim that this question has not been studied elsewhere.

2.5 Imitation learning and DAgger

Training a model to copy an expert from a fixed set of expert decisions is called behavior cloning. Its weakness is distribution shift: once the model makes a mistake, the cache reaches a state the expert never visited, and the model has no training data for it. **DAgger** (dataset aggregation) [8] fixes this by letting the learner drive. The current model runs the system, the expert labels the states that the learner actually visits, these labeled states are added to the training data, and a new model is trained on the union. The process repeats.

In this project the expert is Belady. A round of DAgger replays a training trace with the learned policy in control of the cache and records, at each eviction, the lines that Belady would have evicted in that same state. Only training and validation traces are used, never test traces (Figure 3.1).

2.6 Model families

- **MLP.** A small multi-layer perceptron scores each of the 16 candidate lines from numeric features and two embeddings of program-counter (PC) ids. The line with the

highest score is evicted. The training loss is a softmax over the set against the Belady victim set.

- **Gradient-boosted decision trees (LightGBM)** [9]. A binary classifier per candidate (is this way a Belady victim). It is large in bytes but needs few operations per decision.
- **LSTM with attention** [10], [16]. An LSTM reads the last 16 LLC accesses; each candidate line attends over the hidden states and receives a score. It is the most expensive model, because 16 LSTM steps are needed for every decision.

2.7 Several applications sharing a cache

When programs are time-sliced or run on different cores, their lines mix in the same sets. A simple policy decides without looking at who owns a line. A per-application model, however, has been trained on one program, and in the shared cache it faces lines of other programs whose PCs it has never seen. Our experiments test three ways to combine models, called **arbitration rules** (Section 3.3.2): the requester's model chooses among all lines (req), a model may only evict its own application's lines (own), or one generic model decides for everybody (gen). Fairness between applications is measured with the Jain index [17].

Chapter 3
Problem Statement / Requirement Specifications

Problem statement. Last-level cache replacement is far from optimal: on our workloads the best simple policy leaves a large part of the gap between LRU and Belady open. Learned policies that imitate Belady can close part of it, but it is not clear (a) whether this works on inputs that were not used in training, (b) whether each application should carry its own model or one generic model is enough, (c) what such models cost in storage, decision time and activation, and (d) whether models trained on one application remain useful when several applications share the cache. This project designs and builds a complete simulation environment, trains and evaluates the models, and reports honestly where they help and where they fail.

3.1 Project Planning

3.1.1 Research questions and objectives

1. **RQ1.** Does a per-application learned policy close a meaningful share of the LRU-to-Belady gap on held-out inputs of the same application?
2. **RQ2.** Does a per-application model beat one generic model of equal size and one of equal total storage?
3. **RQ3.** What do the carried models cost: storage, software inference time and activation or switching cost?
4. **RQ4.** In a shared LLC, how do independent models interact under different arbitration rules, and can training on the shared stream (mixed-stream DAgger) improve them?

The objectives that follow from these questions are: build a trace-driven simulator with the standard baselines and Belady; generate Belady-labeled training data; train and compare three model families with DAgger; run the experiments E1 to E8 on held-out test inputs; run two extensions (LSTM DAgger and mixed-stream DAgger); and deliver a report, a static web simulation and a reproducible repository.

3.1.2 Work plan

The work was done in stages. Each stage ended with tests and with results that were recorded in a project log before the next stage began (Table 2).

Table 2: Work plan and status of the project stages

Stage	Content	Result
A	Environment, Valgrind tracing, L1/L2 filter, simulator with LRU, FIFO, Random, SRRIP, Belady, unit tests	Done; 4 workloads, repeatable traces
B	BIP, BRRIP, DRRIP; Belady labeler; dataset format	Done
C	Features, MLP and LightGBM training, C++ inference, parity tests	Done
D	MRU baseline, LSTM with attention, DAgger rounds, validation comparison	Done
E, F	Model selection on validation, then one run on the test split: experiments E1 to E8	Done; test split used once, model set frozen
Ext. 1	LSTM with one DAgger round	Done; one test run
Ext. 2	Mixed-stream DAgger for MLP, LightGBM, LSTM; round 2	Done; one test run on a second mixture
G	Static web simulation	Built and checked
H	This report, then the repository	Report in preparation; repository after the report

3.2 Project Analysis (Requirement Specification)

3.2.1 Functional requirements

- **FR1** Trace four real programs and keep only the memory accesses of the region of interest.
- **FR2** Simulate L1, L2 and a configurable LLC and replay LLC traces under any policy; report hits, misses and MPKI.
- **FR3** Implement LRU, FIFO, Random, MRU, SRRIP, BIP, BRRIP, DRRIP and Belady.
- **FR4** Produce one training row per eviction with past-only features and the Belady victim set as label.
- **FR5** Train MLP, LightGBM and LSTM models, specialized (one application) and generic (all), and run them inside the simulator.
- **FR6** Run DAgger rounds with the learned policy in control of a single-application cache and of a shared cache.
- **FR7** Simulate time slicing and a shared LLC for several applications under the rules req, own and gen.
- **FR8** Measure model size, operations and software decision time, and compute activation cost with an explicit cost model.
- **FR9** Provide a static web simulation that shows real results and sample replays.

3.2.2 Non-functional requirements

- **Honest evaluation.** Training uses train inputs, model choices use validation inputs, and every test input is run once. Marker files refuse a second test run.
- **No future information in the model.** The model sees the current access, the contents of the set and the past. The next use of a line is label information only.
- **Reproducibility.** Fixed program inputs and seeds, ASLR off, empty environment, configuration hash in every result, one command per pipeline stage.
- **Verification.** Every component has unit or parity tests; the C++ model inference must reproduce the Python model decision by decision.

3.2.3 Risks and how we handled them

Table 3: Main risks and the measures taken

Risk	Measure
Training data from the cache of the expert differs from the states the learner creates (distribution shift)	DAgger rounds; mixed-stream DAgger for the shared cache
Choosing models or settings on the test split inflates results	Selection rule on validation only; model set frozen with SHA-256 checksums; marker files block repeated test runs
Matrix and conv are easy: a learned policy could look good by copying MRU	MRU added as a baseline; results always shown against the best simple policy
Software latency is not hardware latency	Reported as software numbers with the 100 ns hardware budget next to them
One trace per application does not represent real programs	Four different access patterns, two test inputs per application, different data shapes; limits stated in Section 4.4
Long jobs killed by memory or by laptop sleep	Resumable scripts; generic-model training limited to two parallel jobs because it needs about 4 GB of RAM each

3.3 System Design

3.3.1 Design Constraints

Hardware and software. All work was done on one personal laptop (Windows) using WSL2 with Ubuntu. Tools: a C++17 compiler for the simulator, Valgrind [11] for tracing, Python with PyTorch [12] for MLP and LSTM training and LightGBM [9] for trees. No GPU was used; all models are small. The complete results of the project take about 27 GB of disk space.

Simulation assumptions. We state these because they limit what the results mean (they are fixed in the decision log of the project).

- Only data accesses are cached; instruction fetches are not. Stores are treated like loads; dirty write-backs from L2 to the LLC are not modelled.
- L1 and L2 use LRU, are non-inclusive and write-allocate. The LLC fetches on demand and never bypasses.
- Addresses are virtual. There is no prefetcher, no timing model and a single core; applications are interleaved in time by a round-robin schedule.
- The first 10% of every LLC trace warms the cache; its misses are not counted.
- Belady ties (several lines never used again) are broken by the lowest way index in the simulator and are all accepted as correct labels during training.

Table 4: Cache configuration used in all experiments

Level	Size	Associativity	Lines	Sets	Policy
L1 data	32 KB	8-way	512	64	LRU
L2	256 KB	8-way	4,096	512	LRU
LLC (default)	2 MB	16-way	32,768	2,048	policy under study
LLC (other)	1 MB / 4 MB / 8 MB 16-way; 2 MB 8-way				E5 and E8 (8 MB)

Line size 64 B at all levels.

3.3.2 System Architecture / Block Diagram

Figure 1.1 shows the data flow. The components and their main files are listed in Table 5. Everything that touches the cache is written in C++ (simulator, labeler, multi-application engine, exporter); training is in Python.

Table 5: Components of the system

Component	Task	Main file
Workloads	Four C programs with fixed inputs and a region-of-interest marker	applications/*.c
Tracing	Valgrind lackey trace, converted to a compact binary trace	trace_collection/, lackey2trace
Cache simulator	L1/L2 filter; LLC replay under any policy; learned policies run here	simulator/cachesim.cpp, learned.h
Belady labeler	Replays a trace with a driver policy and records Belady victim sets	belady/labeler.h
Training	Features, MLP, LightGBM, LSTM, DAgger rounds, model export	training/*.py
Multi-application engine	Round-robin schedule, ownership, rules req / own / gen, per-application statistics	simulator/multi.h, multisim.cpp
Experiment drivers	Selection rule, E1 to E8, tables, plots, cost model	scripts/run_final.sh, final_*.py

Component	Task	Main file
Web simulation	Static page with results and sample-window replays	web_simulation/, export/

Training data: what one row contains

One row is one **eviction decision**: a miss in a set whose 16 ways are all valid. For each of the 16 candidate lines the row holds eight numbers computed from the past: time since the last access, age since insertion, access count, a re-use flag, the last gap between accesses, the recency rank in the set, whether the last access was a store, and the id of the PC of the last access. It also holds the PC id and the store flag of the current access. PC ids come from a vocabulary of the 64 most frequent training PCs; 0 means unknown. The label is the set of ways that Belady would evict (several if some lines are never used again). Belady's next-use index is stored with the row, but only for labeling and analysis; it is never a model input.

DAgger and the arbitration rules

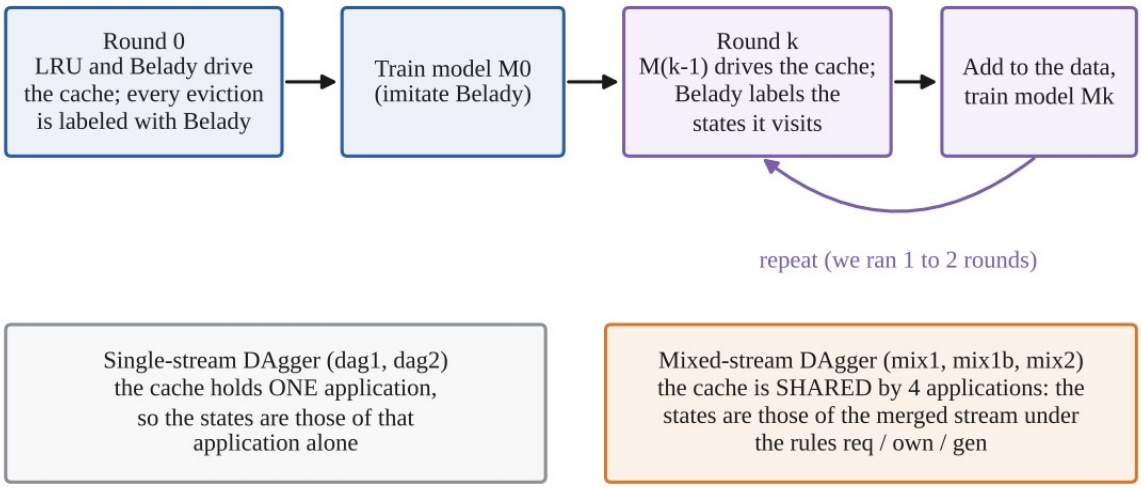


Figure 3.1: DAgger rounds. In single-stream DAgger the cache holds one application; in mixed-stream DAgger the cache is shared by four.

Table 9 in Section 4.1.5 names the data sets by the policy that drove the cache when they were recorded. For the shared cache we use three rules to decide who picks the victim when application R misses in a full set (Figure 3.2). Under **req**, R's own model scores all lines of the set and may evict any of them, also lines of other applications. Under **own**, each application is allowed a quota of ways / applications = 4 lines per set; if R owns at least the quota, R's model must evict one of its own lines, otherwise the application with the most lines in the set gives up one line, chosen by that application's model. A model never chooses among lines of another application. Under **gen**, one generic model decides for everybody.

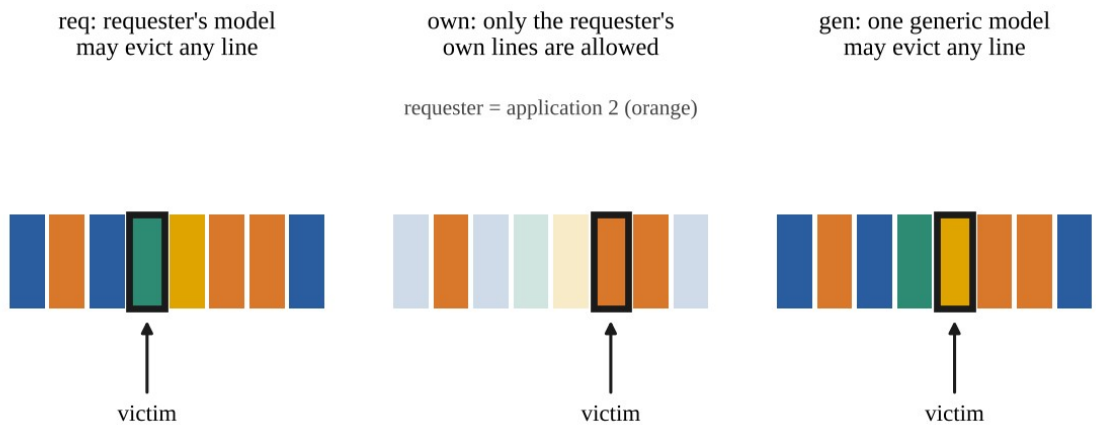


Figure 3.2: The three arbitration rules for one set (illustration; colours are owning applications and the requester is the orange application).

The application-carried model and its activation cost

In the application-carried design each program has a small model file that travels with it. When the scheduler switches to application B, the cache controller loads B's model. We do not build hardware for this; we compute what it would cost with an explicit model: T = T_fixed + bytes / bandwidth if the model is loaded on demand and T = T_fixed if all models are already resident. The parameters (miss penalty 80 ns, T_fixed 10 µs, bandwidth 10 GB/s, 0.33 ns per instruction) are assumptions and are swept in the results. The activation cost is a model output, never a measurement.

Chapter 4
Implementation

4.1 Methodology / Proposal

This section describes what was built, in the order in which data moves through the system. Every step has a command-line program and a test, so the whole chain can be repeated.

4.1.1 Workloads and inputs

We wrote four small C programs whose memory behavior is typical and different from each other. The memory accesses of the region of interest (ROI) are cut out with start and end markers so that start-up code is not traced.

Table 6: The four workloads

Workload	What it does	Access pattern
hash	Open-addressing hash table with 2^18 slots; 100,000 keys and 1.5 million lookups; keys drawn uniformly or skewed towards a hot part	Random accesses with a hot part
graph	Breadth-first traversals of a random graph of 100,000 nodes and average degree 8; uniform or hub-heavy edges	Pointer chasing; irregular reuse
matrix	Dense matrix multiplication, N = 500 to 800	Strided sweeps larger than the LLC
conv	2-D convolution over an image of width 640 to 832 with a K × K filter	Sliding window with a streaming part

Each application has three training inputs, one validation input and two test inputs (24 runs in total). The test inputs change the data shape, not only the random seed: for hash the key distribution, for graph the hub structure, for matrix N = 700 and 800, for conv the image width and the filter size. The full list of arguments is part of the configuration file `configs/inputs.tsv` of the repository. Inputs are **never** exchanged between splits.

Table 7: Inputs per split (full arguments in configs/inputs.tsv)

Split	Runs per application	Used for
train	3	Training data and DAgger rounds
val	1	All choices: model family version, DAgger rounds, hyper-parameters
test	2	One run of the frozen models; reported results

4.1.2 Tracing and the L1/L2 filter

Valgrind’s lackey tool [11] records every load, store and instruction address. To make the traces repeatable we switch address-space randomization off, run the program with an empty environment and keep only the ROI. A converter writes a compact binary trace. The simulator then passes the trace through a 32 KB L1 and a 256 KB L2 (Table 4); the accesses that miss in L2 form the **LLC trace**. This matters, because the LLC sees a stream already filtered by the upper levels, and it is that stream on which replacement is judged. The first 10% of the LLC trace warms the cache and is not counted.

4.1.3 Belady labeler and dataset

The labeler replays an LLC trace. At every miss in a set whose 16 ways are valid it writes one record: the features of the 16 lines, the current access, and the set of ways that Belady would evict. Belady needs the next use of every line; this value is computed by a backward pass over

the trace and is saved only as label information. The record layout is fixed (see `docs/dataset_format.md` in the repository). Only **informative** decisions are used for training: if all candidates are never used again, or if all would be equally good, there is nothing to learn from the row.

The labeler can run with different **drivers**, meaning different policies that control the cache while the labels are recorded. With LRU as the driver, the data come from the cache states that LRU creates. With a learned model as the driver (DAgger), the data come from the states the learner creates. Labels are always the Belady victims for that state.

4.1.4 Models

Table 8: Model families as built

Model	Input per candidate line	Output	Training
MLP	8 numeric features; 2 PC embeddings (64 ids); current-access features	A score per line; evict the highest	Softmax over the set, cross-entropy against the Belady victim set
LightGBM	Same features as numbers (PC ids as integers)	Probability that the line is a Belady victim; evict the highest	Binary classification with early stopping on validation
LSTM + attention	The same line features and PC embeddings, plus attention over the last 16 LLC accesses (PC id, line-address change, store flag)	A score per line; attention over the history states	Softmax cross-entropy plus an auxiliary loss on the log reuse distance (dropped at export)

The LSTM is inspired by PARROT [6] but is much smaller (history 16, LSTM size 32, embedding size 4, about 9,400 parameters) and is applied stateless: for each decision the history is replayed from the last 16 accesses. PARROT uses a larger network and a longer history. All models are exported to a plain binary format and executed in C++ inside the simulator, so the learned policy runs in the same loop as LRU or DRRIP.

Specialized and generic models. A specialized model is trained on one application. A generic model is trained on the data of all four applications. For a fair comparison with equal storage we also train a larger generic MLP (“generic-big”, 7,495 parameters) whose size equals that of the four specialized MLPs together.

4.1.5 DAgger variants and what drives each data set

Table 9: Data sets and models by the policy that drove the cache

Name	Driver while recording	Cache seen by the driver	Used for
base	LRU	one application	First model of each family
dag1	Base model	one application	Round 1 of single-stream DAgger (MLP, GBDT; LSTM extension)
dag2	dag1 model	one application	Round 2; it gave no consistent gain
mix1	Frozen single-application models (req and own rule) and the frozen generic model	four applications sharing the LLC	Round 1 of mixed-stream DAgger
mix1b	mix1 data plus data driven by LRU and by Belady in the shared cache	four applications sharing the LLC	Adds states created by the experts
mix2	mix1 model	four applications sharing the LLC	Round 2 of mixed-stream DAgger

In mixed-stream DAgger the merged access stream of the four applications is produced by the round-robin schedule of the multi-application engine. Owner ids sit in the upper bits of the line tag. The PCs of other applications are hidden from a model (id 0), so a model cannot read what

the other programs are doing. The training mixtures use training inputs only; the test mixtures use test inputs.

4.1.6 Experiments

Table 10: Experiments

ID	Question	Setting
E1	How good are the simple policies?	All baselines on the test traces
E2	Do learned policies beat the best simple one?	MLP, GBDT, LSTM, specialized and generic, test traces
E3	Specialized or generic?	Equal size and equal total storage; 3 seeds for the MLP
E4	Do models transfer between applications?	Every model on every application
E5	Do models survive other LLC configurations?	1 MB, 4 MB, 2 MB 8-way, no retraining
E6	What do the models cost?	Size, operations, decision time, activation cost
E7	What does switching cost?	Four applications in turn; slices of 40 k to 10 M instructions
E8	What happens in a shared LLC?	Four applications in round-robin; req, own, gen; 2 MB and 8 MB
X1	Extension: does DAgger help the LSTM?	One DAgger round; one test run
X2	Extension: can mixed-stream DAgger fix the shared cache?	mix1, mix1b, mix2; validation, then one test run on a second mixture

4.2 Testing / Verification Plan

A result is only worth as much as the code that produced it, so every component was tested before it was used for experiments. Table 11 lists the test cases; the `make test` command runs all of them. The numbers of checks are from the last complete run.

Table 11: Test cases

Test ID	Test case title	Test condition	System behavior	Expected result
T1	Policy hand checks	Small traces whose result was worked out on paper for LRU, FIFO, SRRIP, BIP, DRRIP	Simulator replays the trace	Miss counts equal the hand-computed values
T2	Belady optimality	Random small traces; exhaustive search over all victim choices	Belady replay	Belady misses equal the minimum, and are not above any other policy
T3	Next-use and warm-up	Traces with known reuse distances	Next-use pass; warm-up handling	Values equal brute force; warm-up misses not counted
T4	Random determinism	Same seed twice	Random policy	Identical output
T5	Labeler	Hand-checked small trace; random traces	Labeler writes rows	Label equals Belady set; no feature uses the future
T6	Filter and trace I/O	Known access sequences; lackey text samples	L1/L2 filter; trace reader and writer	Correct filtered stream; round trip equal
T7	Model parity	Trained MLP and GBDT, thousands of decisions	C++ inference versus Python	Same victim every time (0 mismatches); LSTM within tolerance
T8	Dataset	Brute-force next-use on a small trace	Python dataset builder	Equal to the brute-force value
T9	DAgger	Small driver and trace	One DAgger round	Data driven by the learner; labels still Belady
T10	Multi-application engine	Round-robin schedule, ownership, rules req/own/gen	multisim	Isolation, correct owner, quota, single-application equivalence
T11	PC masking	Mutation test: the mask removed on purpose	multisim with masked PCs	Mutated run must fail the test
T12	Mixed pipeline	Whole mixed-stream chain on a tiny trace	Labeler versus multisim	Same miss count; no test split touched

Test ID	Test case title	Test condition	System behavior	Expected result
T13	Final tables	Selection rule recomputed; tables compared with a direct run	run_final	Match; frozen set is not rewritten; a failing job is reported
T14	Web data	Window exporter against the simulator; data packer	winexport, build_web_data	Equal counts; no future field in the page data

The C++ unit-test program (`run_tests`) makes 2,983,251 checks and the multi-application test program (`run_multi_tests`) 4,872 checks. All Python tests pass. The tests do not prove that the experiments are right in every respect: they show that the simulator, labeler and model code do what they are supposed to do, and that the data rules (no test data in training, no future in the model) are enforced by code and not only by discipline.

4.3 Result Analysis / Screenshots

All numbers below are computed from the result files of the experiments; none was typed by hand. “Gap closed” is explained in Section 2.3. Unless stated otherwise, the LLC is 2 MB, 16-way, and every figure is for the held-out test inputs.

4.3.1 E1 and E2: learned policies against baselines

On hash and graph the simple policies leave a large gap: the best of them (BIP) closes 35.3% and 22.7%. On these programs FIFO and Random are worse than LRU, and MRU is far worse (-81% to -449% on the gap scale). Learned models do clearly better. The MLP closes 46% to 47% on hash and 36% on graph, so it removes about one third more of the gap than the best simple policy on hash, and over a half more on graph. On matrix, the best simple policy (DRRIP) already closes 96.4%, and all learned policies reach 100%; on conv the best simple policy closes 53.5% and the MLP 100%. GBDT is close to the MLP on graph and a few points lower on hash. The LSTM with attention is behind the MLP, and on conv the generic LSTM closes only 86.4%.

Table 12: E2: mean gap closed on the two test inputs of each application

Application	Best simple	MLP spec.	MLP generic	GBDT spec.	GBDT generic	LSTM spec.	LSTM generic
hash	35.3%	46.3%	47.1%	41.0%	40.7%	39.2%	36.6%
graph	22.7%	35.8%	36.2%	34.9%	34.2%	32.9%	33.0%
matrix	96.4%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
conv	53.5%	100.0%	100.0%	99.5%	99.8%	100.0%	86.4%

Best simple = the best of LRU, SRRIP, DRRIP, BIP and MRU on that trace. Spec. = specialized, one model per application; generic = one model for all.

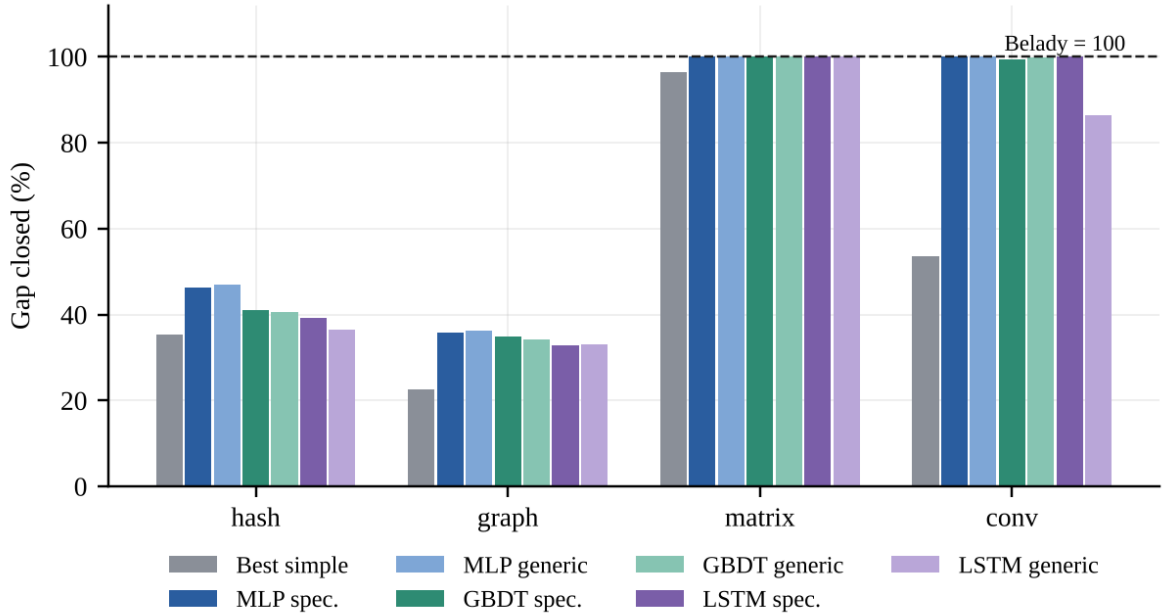


Figure 4.1: Gap closed on the test inputs, learned models against the best simple policy.

Matrix and conv are easy for learned policies because the access patterns are regular. The clearest value is therefore on hash and graph, where the MLP has 6% to 9% fewer misses than the best simple policy on each of the four test inputs. We caution that both are single, synthetic programs.

4.3.2 E3: specialized or generic?

Table 13 shows the MLP on the hash and graph test inputs, with three training seeds. The four specialized MLPs together do **not** beat one generic MLP of the same size as one specialized model. The generic model is better by 0.2 to 0.9 points on these inputs, and the larger generic model of equal total storage is no better than the small one. On matrix and conv every variant reaches 100% on both inputs, so they cannot separate the variants. Specialization therefore showed no advantage in accuracy in our setting, and it needs 3.3 to 4.6 times more storage (Table 17).

Table 13: E3: gap closed (mean ± standard deviation over three seeds), MLP

Test input	Specialized (4 models)	Generic, equal size	Generic, equal total storage
hash, uniform keys	44.7 ± 0.2	45.4 ± 0.6	45.1 ± 0.5
hash, skewed keys	47.7 ± 0.2	48.6 ± 0.5	48.3 ± 0.4
graph, uniform	41.8 ± 0.2	42.1 ± 0.0	42.2 ± 0.0
graph, hub-heavy	29.3 ± 0.3	30.1 ± 0.3	30.2 ± 0.3
matrix N=700	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
matrix N=800	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
conv W=720, K=5	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
conv W=832, K=3	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0

Matrix and conv: 100.0 ± 0.0 for all three variants on all four test inputs.

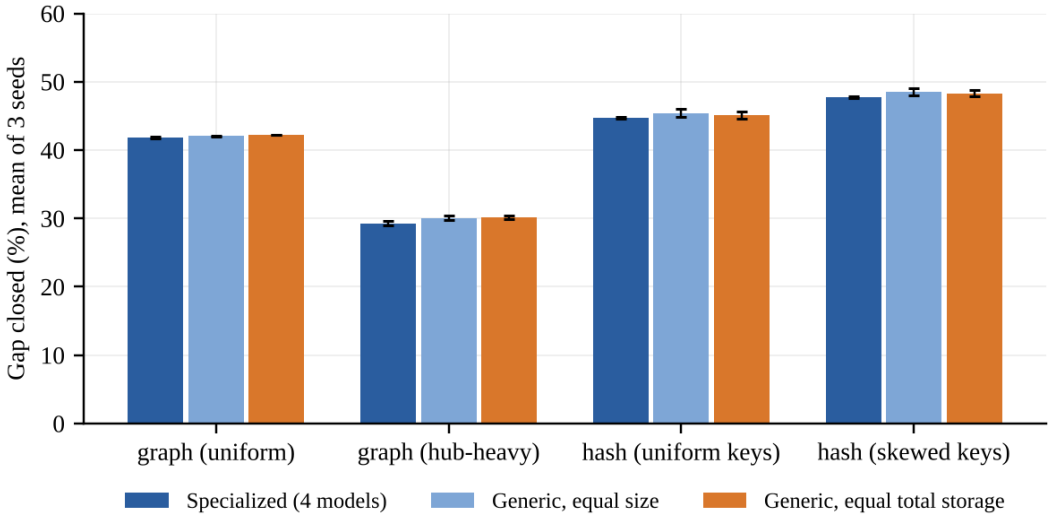


Figure 4.2: Specialized and generic MLPs on the hash and graph test inputs.

4.3.3 E4: models do not transfer between applications

In E4 each model is run on each application. On its own application it works. On another application it can be much worse than LRU: the graph model on hash closes −143.9%, and the models of matrix and conv close −374.7% and −425.3% on graph. A plausible reason is that a rule that suits one pattern (for example, keep old lines in a sweep) is harmful on another. This is why a single application model cannot simply be left in a shared cache.

Table 14: E4: gap closed, model of one application (rows) run on another (columns)

	on hash	on graph	on matrix	on conv
model of hash	46.3%	15.4%	91.9%	0.0%
model of graph	-143.9%	35.8%	0.0%	99.6%
model of matrix	-110.8%	-374.7%	100.0%	100.0%
model of conv	-110.4%	-425.3%	93.7%	100.0%
generic model	47.1%	36.2%	100.0%	100.0%

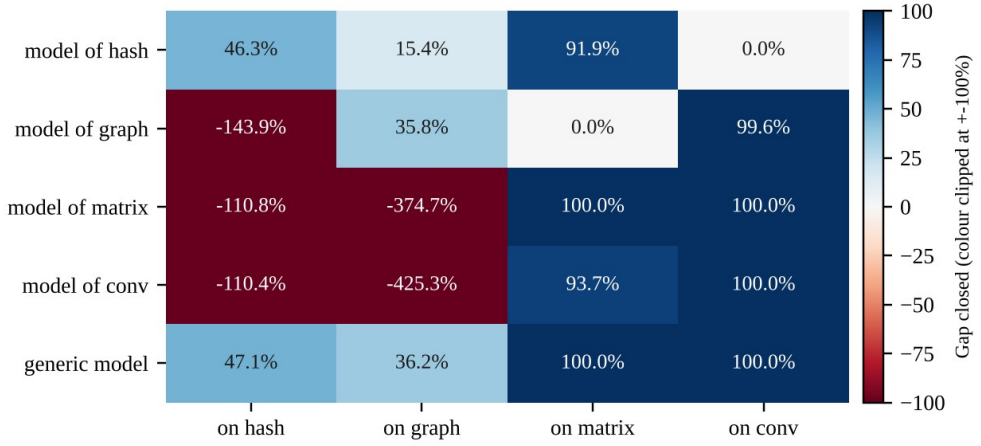


Figure 4.3: Cross-application results (E4).

4.3.4 E5: other LLC configurations without retraining

Models trained for the 2 MB LLC were run unchanged on a 1 MB LLC, on a 2 MB 8-way LLC and on a 4 MB LLC. They still beat the best simple policy on every application in every

configuration. At 4 MB the hash trace has only a small gap to close (best simple 17.1%, MLP 24.0% to 26.2%); on graph the MLP closes 69.4% to 69.8% against 38.8% for the best simple policy. The result shows that the models did not learn the exact cache size, but it is a limited test: the number of sets changes in all three configurations, but the line size and the programs stay the same.

Table 15: E5: gap closed in other LLC configurations (selected rows)

LLC configuration	Application	Best simple	MLP spec.	MLP generic
1 MB, 16-way	hash	37.1%	50.6%	50.5%
1 MB, 16-way	graph	43.7%	62.7%	62.8%
1 MB, 16-way	matrix	95.4%	100.0%	99.9%
1 MB, 16-way	conv	56.8%	100.0%	100.0%
2 MB, 8-way	hash	33.3%	47.3%	48.3%
2 MB, 8-way	graph	20.4%	37.3%	37.6%
2 MB, 8-way	matrix	93.7%	100.0%	99.5%
2 MB, 8-way	conv	55.0%	100.0%	100.0%
4 MB, 16-way	hash	17.1%	24.0%	26.2%
4 MB, 16-way	graph	38.8%	69.4%	69.8%
4 MB, 16-way	matrix	97.7%	100.0%	99.7%
4 MB, 16-way	conv	69.5%	100.0%	100.0%

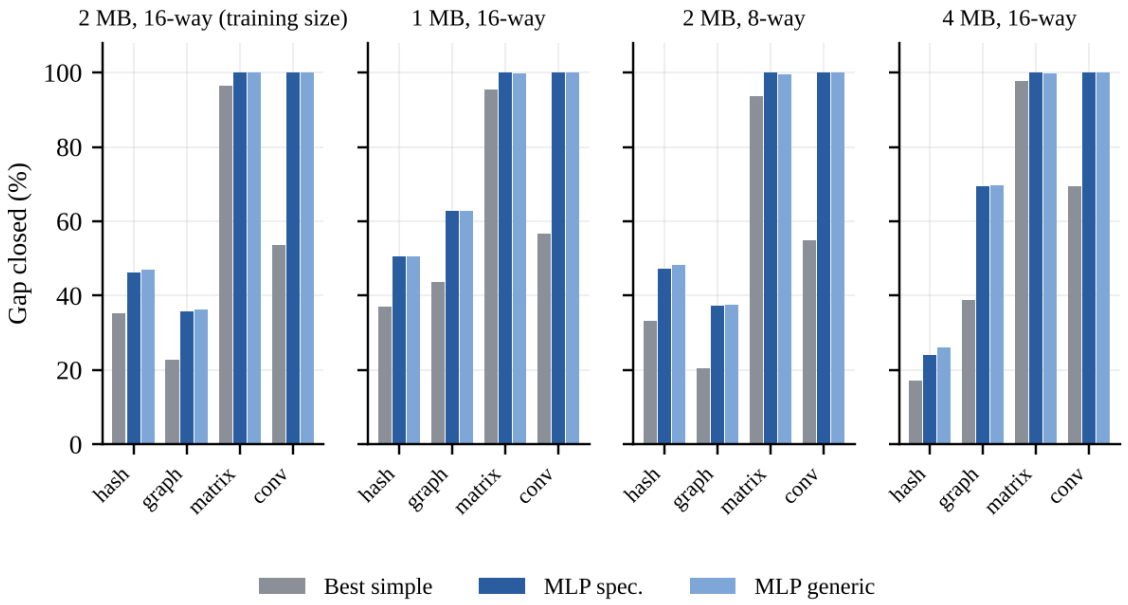


Figure 4.4: Gap closed under other LLC configurations.

4.3.5 E6: cost of storing and running the models

Table 16 shows the size, the number of operations per decision and the software time per decision. The MLP needs 7 to 9 KB and 8 to 12 µs at the median. Trees are large (155 KB to 1.06 MB for the specialized GBDT models) and need fewer operations but memory-bound time (7 to 146 µs). The LSTM needs about 38 KB and 100 to 134 µs, because 16 recurrent steps run for every decision. These are software numbers on one laptop core. A hardware eviction path has a budget of about 100 ns, so every model here is at least two orders of magnitude too slow, and the results should be read as an upper limit on what a learned policy could achieve with a cheaper design, not as a design.

Table 16: E6: model cost

Model	Params	Bytes	Ops / decision	p50 (us)	p99 (us)
specialized mlp (hash)	1,739	7,044	52,528	8.4	14.6
specialized mlp (graph)	2,203	9,364	52,528	12.1	39.0
specialized mlp (matrix)	1,731	7,004	52,528	10.4	18.5
specialized mlp (conv)	1,739	7,044	52,528	10.8	21.9
generic mlp	2,203	9,364	52,528	11.1	26.4
generic-big mlp	7,495	30,532	220,528	46.3	134.0
specialized gbdt (hash)	18,300	1,058,844	16,762	146.0	441.8
specialized gbdt (graph)	13,237	769,533	16,921	125.0	354.4
specialized gbdt (matrix)	3,050	179,814	5,472	13.1	28.3
specialized gbdt (conv)	2,623	154,851	3,161	7.4	16.8
generic gbdt	7,869	471,637	12,889	53.9	130.7
specialized lstm (hash)	9,363	37,540	325,632	111.1	364.8
specialized lstm (graph)	9,595	38,932	325,632	114.0	308.3
specialized lstm (matrix)	9,359	37,516	325,632	134.2	604.0
specialized lstm (conv)	9,363	37,540	325,632	103.7	300.6
generic lstm	9,595	38,932	325,632	100.7	365.7

Ops = multiply-accumulate and compare operations per eviction decision; p50 and p99 = median and 99th percentile of the software time per decision on one core.

Table 17: E6: storage of four specialized models against one generic model

Family	4 specialized (B)	Generic (B)	Generic-big (B)	4 specialized / generic
MLP	30,456	9,364	30,532	3.3x
GBDT	2,163,042	471,637	-	4.6x
LSTM	151,528	38,932	-	3.9x

Activation cost follows from the cost model of Section 3.3.2. With models resident in the cache controller, a switch costs only T_fixed. If a model must be loaded when the application is scheduled, the load time dominates: for the GBDT models (mean 540,760 bytes) the transfer time at 1 GB/s is 541 µs, against 8.6 µs for the MLP. Table 18 shows the activation time for the mean model size of each family.

Table 18: E6: activation time per switch (µs) for the mean model size of each family

Family	Mean bytes	Bandwidth (GB/s)	T_fixed 1 us	T_fixed 10 us	T_fixed 100 us
MLP	7,614	1	8.61	17.61	107.61
MLP	7,614	10	1.76	10.76	100.76
MLP	7,614	100	1.08	10.08	100.08
GBDT	540,760	1	541.76	550.76	640.76
GBDT	540,760	10	55.08	64.08	154.08
GBDT	540,760	100	6.41	15.41	105.41
LSTM	37,882	1	38.88	47.88	137.88
LSTM	37,882	10	4.79	13.79	103.79
LSTM	37,882	100	1.38	10.38	100.38

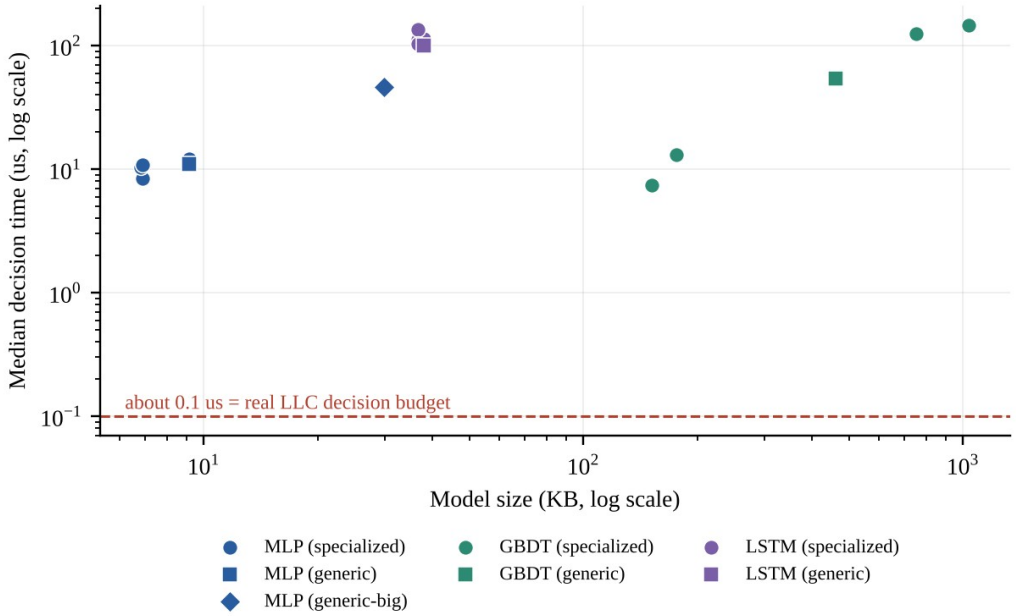


Figure 4.5: Model size against software decision time (E6).

4.3.6 E7: switching between applications

We interleaved the four applications by time slices of 40 k, 400 k, 4 M and 10 M instructions; the number of switches in the run goes from 3,997 to 13. For the specialized per-application models (req rule) the results are below the best simple policy at every slice length. Table 19 gives the miss changes against LRU. With short slices, the cache is refilled after every switch, and the learned models are at or below LRU (40 k: −6.3% to +0.2% for the generic models, −17.2% to −14.5% for the specialized). At 400 k, the generic GBDT reaches +19.9% but DRRIP has +25.9%. At 4 M and 10 M no policy is clearly better than LRU, and the learned policies are 4% to 51% worse than LRU. No learned policy beat the best simple policy at any slice length. The break-even analysis agrees: at decision latencies of 20, 100 and 1000 ns the time saved against the best simple policy is negative, so the answer to “when does a per-application model pay for its switch?” is “never (net loss)” for the frozen models.

Table 19: E7: LLC misses against LRU (positive = fewer misses), by slice length

Policy (vs LRU)	40 k slice	400 k slice	4 M slice	10 M slice
LRU (misses)	**2,481,450**	**2,612,880**	**1,689,412**	**1,442,700**
SRRIP	+8.1%	+12.5%	-0.8%	+1.5%
DRRIP	+18.8%	+25.9%	-0.9%	+0.9%
BIP	+15.8%	+24.1%	-14.7%	-36.6%
MRU	-59.9%	-51.1%	-128.6%	-162.9%
generic GBDT	-6.3%	+19.9%	-10.8%	-21.8%
generic MLP	+0.2%	+9.2%	-37.0%	-44.2%
generic MLP-big	-1.7%	+7.5%	-37.2%	-51.2%
specialized GBDT (req)	-14.5%	+10.2%	-3.9%	-13.7%
specialized MLP (req)	-17.2%	+7.8%	-10.8%	-10.3%
Belady	+36.3%	+40.5%	+21.2%	+33.8%

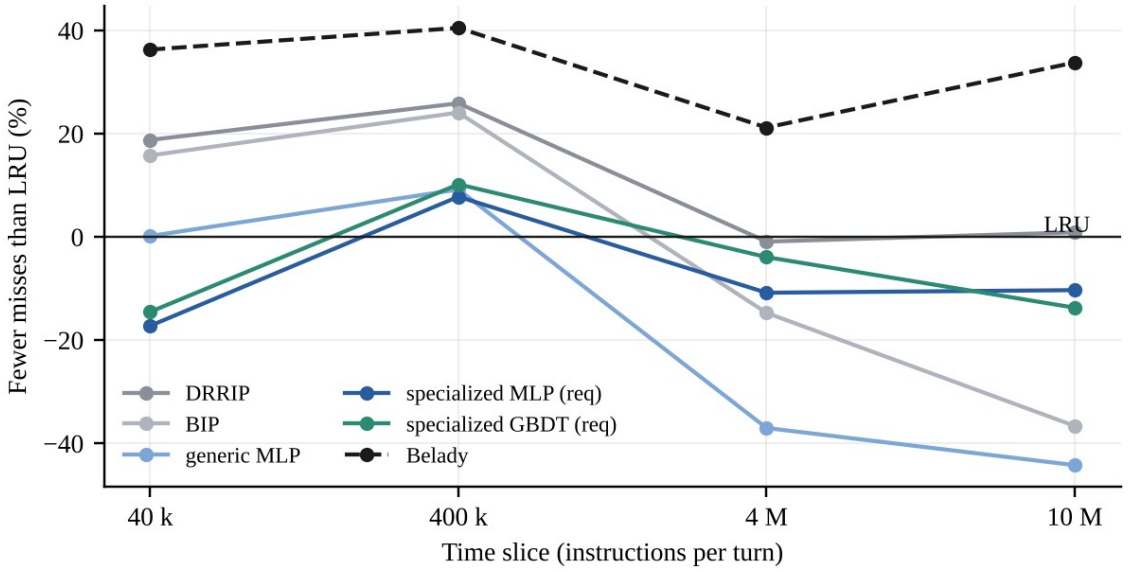


Figure 4.6: Switching: effect of the slice length (E7).

4.3.7 E8: four applications in one shared LLC

Four applications run in round-robin through a 2 MB LLC (hash, graph, matrix and conv together in one mixture; the table shows the MPKI of three of them and the fairness). DRRIP is the best simple policy and reduces misses by 18.5% against LRU; Belady reduces them by 33.2%. The generic MLP gets 6.3% and the specialized MLPs under the **own** rule 5.8%, both below DRRIP. Under the **req** rule, where every application's own model can evict any line, the specialized MLP has 15.8% **more** misses than LRU: the model of one application evicts the lines of the others by rules learned on its own program. The Jain index (1.0 is perfectly fair) is 0.85 to 0.96; Belady is the fairest at 0.958 and DRRIP follows at 0.930.

Table 20: E8: shared 2 MB LLC, four applications

Policy	Misses	vs LRU	MPKI hash	MPKI graph	MPKI matrix	Jain fairness	Cross-app evictions
LRU	3,229,206	-	28.99	24.64	17.88	0.855	67.5%
SRRIP	2,978,663	+7.8%	26.15	20.49	17.88	0.871	66.8%
DRRIP	**2,633,063**	**+18.5%**	**22.96**	**18.85**	**15.41**	**0.930**	**74.0%**
BIP	2,717,589	+15.8%	24.13	20.65	15.09	0.915	74.5%
MRU	4,499,277	-39.3%	31.64	61.87	15.60	0.947	70.7%
gen GBDT	3,012,205	+6.7%	34.11	25.79	11.55	0.852	75.9%
gen MLP	3,024,787	+6.3%	36.30	22.78	12.16	0.861	76.9%
gen MLP-big	2,936,191	+9.1%	36.17	21.33	11.70	0.862	73.3%
own spec. GBDT	3,462,478	-7.2%	27.41	32.72	17.88	0.883	0.0%
own spec. MLP	3,041,762	+5.8%	26.56	25.43	16.03	0.911	0.0%
req spec. GBDT	3,732,374	-15.6%	29.79	37.88	17.88	0.855	92.7%
req spec. MLP	3,738,607	-15.8%	33.14	34.69	17.88	0.850	83.7%
Belady	**2,157,901**	**+33.2%**	**12.50**	**13.13**	**16.91**	**0.958**	**67.2%**

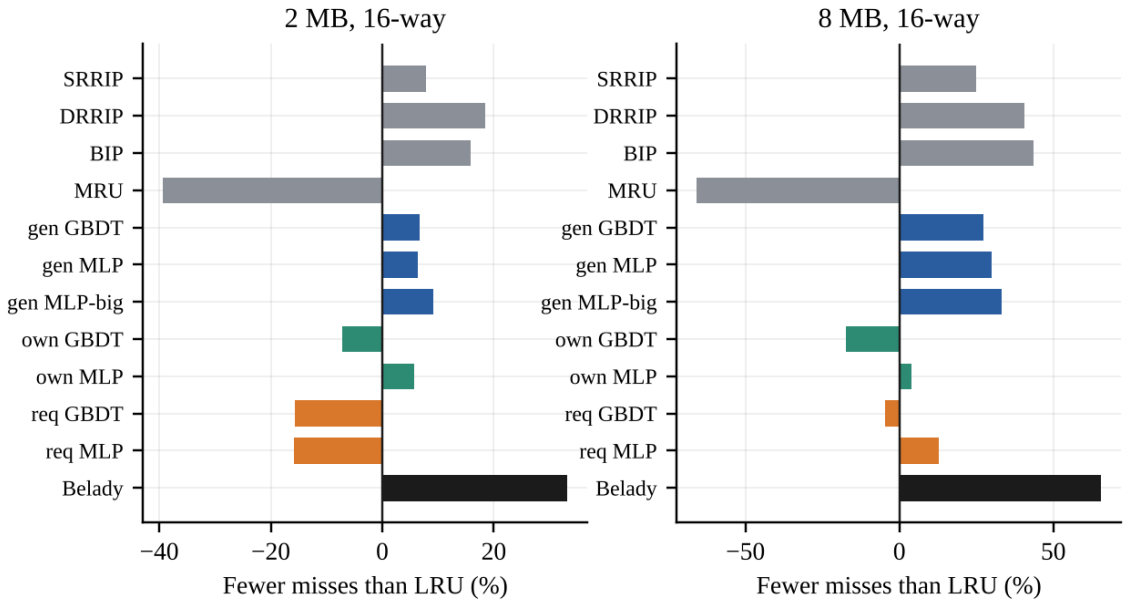


Figure 4.7: Shared LLC results (E8).

4.3.8 Extension 1: DAgger for the LSTM

The first extension asks whether one DAgger round helps the LSTM as it helped the MLP. In the DAgger test (one run, frozen after validation) the mean gap closed over the four applications moves as shown in Table 21. The LSTM improves from 68.0% to 70.6% (specialized) and from 64.0% to 70.8% (generic): the generic LSTM no longer fails on conv. With the same round of DAgger the LSTM equals the MLP in accuracy (70.6 against 70.5 and 70.8 against 70.8) and it costs about five times the storage and ten times the decision time (Table 16). So the LSTM is not worth its cost in our setting.

Table 21: Extension 1: mean gap closed, LSTM DAgger

Model	hash	graph	matrix	conv	Mean (specialized)	Mean (generic)
MLP _dag1	46.3%	35.8%	100.0%	100.0%	70.5%	70.8%
LSTM base	39.2%	32.9%	100.0%	100.0%	68.0%	64.0%
LSTM _dag1	46.4%	36.0%	100.0%	100.0%	70.6%	70.8%

For comparison, the mean gap closed by the simple policies over the same four applications is SRRIP 5.9%, DRRIP 49.1%, BIP 51.2% and MRU −95.7%.

4.3.9 Extension 2: mixed-stream DAgger for the shared cache

The second extension returns to the finding of E8 that models trained on one application at a time fail in the shared cache. We trained all three families on data recorded in the shared cache itself (Table 9). Model versions were chosen on a validation mixture; the test was run once on a **second**, unseen mixture of test inputs. The reference points for this test mixture at 2 MB are LRU 4,325,237, DRRIP 3,583,074, Belady 2,984,374 misses; at 8 MB they are LRU 2,059,635, BIP 978,041, Belady 610,901 misses.

Table 22: Extension 2, test mixture, 2 MB: gap closed / change in misses against the best simple policy (DRRIP)

Family	Version	gen	req	own
MLP	frozen	41.6% / −5.1%	-0.2% / −20.8%	43.4% / −4.5%
MLP	mix1	76.0% / +7.7%	-39.3% / −35.4%	39.7% / −5.9%
MLP	mix1b	77.5% / +8.3%	66.1% / +4.0%	34.4% / −7.8%

Family	Version	gen	req	own
MLP	mix2	72.1% / +6.3%	58.3% / +1.1%	42.2% / −4.9%
GBDT	frozen	56.3% / +0.3%	-4.9% / −22.6%	13.4% / −15.7%
GBDT	mix1b	72.6% / +6.5%	47.3% / −3.0%	31.9% / −8.8%
GBDT	mix2	69.7% / +5.4%	45.4% / −3.7%	33.6% / −8.1%
LSTM	frozen	10.3% / −16.8%	5.3% / −18.7%	16.2% / −14.7%
LSTM	mix1b	78.6% / +8.7%	66.3% / +4.1%	42.9% / −4.6%
LSTM	mix2	76.1% / +7.8%	66.0% / +4.0%	42.7% / −4.7%

Positive change = fewer misses than the best simple policy. frozen = the single-application models of the main experiments; mix1 = mixed-stream round 1; mix1b = mix1 plus data driven by LRU and Belady; mix2 = round 2. mix1 was only run for the MLP.

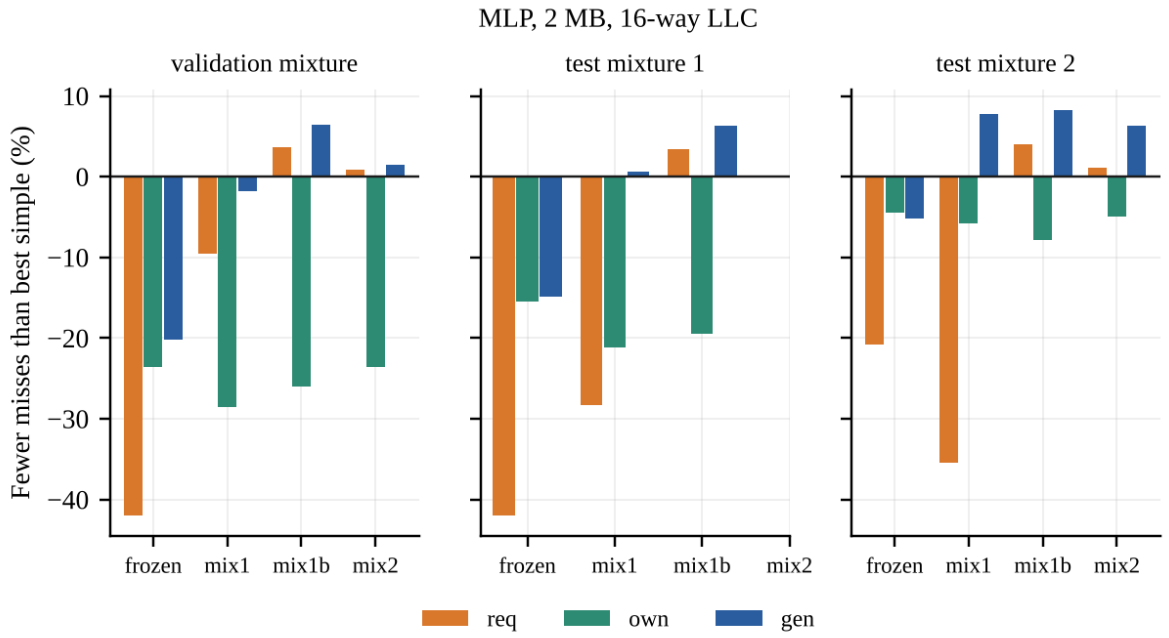


Figure 4.8: Extension 2: results on the test mixture, 2 MB LLC.

At 2 MB the generic models trained on the shared stream beat DRRIP by 5% to 9%: the best are LSTM mix1b (+8.7%), MLP mix1b (+8.3%) and LSTM mix2 (+7.8%). The req rule collapses when it is trained on the wrong stream (MLP mix1: 35.4% more misses than DRRIP, gap closed −39.3%, against 20.8% more before), and recovers with data from the experts (4.0% fewer misses than DRRIP, mix1b). The own rule stays below the best simple policy in every row (−4.5% to −8.8%). The same holds at 8 MB: under own the misses are 65% to 111% above BIP. A likely reason is that the quota forces a model to choose among its own lines only, even when a line of another application would be a better victim.

Table 23: Extension 2, test mixture, 8 MB: gap closed / change in misses against the best simple policy (BIP)

Family	Version	gen	req	own
MLP	frozen	74.7% / +0.0%	39.1% / −52.7%	30.8% / −64.9%
MLP	mix1	84.1% / +14.0%	67.3% / −10.9%	20.6% / −80.0%
MLP	mix1b	67.2% / −11.1%	65.0% / −14.4%	23.1% / −76.3%
MLP	mix2	84.5% / +14.6%	75.1% / +0.7%	26.0% / −72.0%
GBDT	frozen	55.1% / −29.0%	31.3% / −64.3%	-0.1% / −110.8%
GBDT	mix1b	67.8% / −10.1%	69.6% / −7.4%	15.0% / −88.4%

Family	Version	gen	req	own
GBDT	mix2	77.2% / +3.8%	71.7% / −4.4%	21.8% / −78.4%
LSTM	frozen	77.0% / +3.5%	40.7% / −50.3%	29.8% / −66.4%
LSTM	mix1b	78.2% / +5.3%	73.0% / −2.5%	25.5% / −72.8%
LSTM	mix2	85.4% / +15.9%	78.5% / +5.7%	24.4% / −74.4%

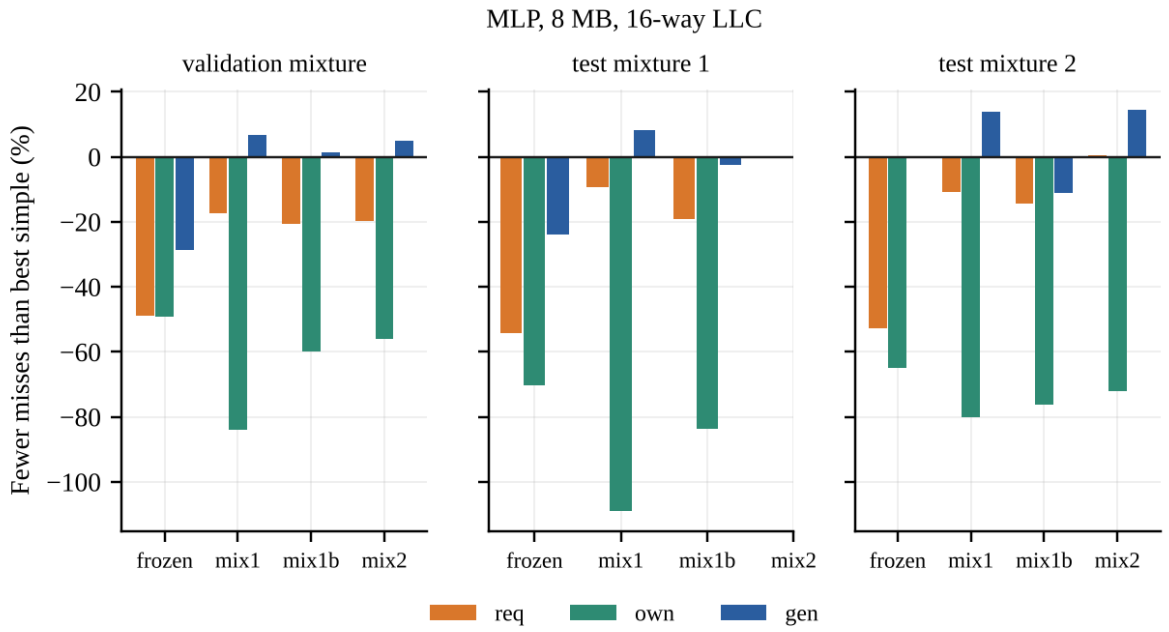


Figure 4.9: Extension 2: results on the test mixture, 8 MB LLC.

At 8 MB the generic LSTM mix2 has 15.9% fewer misses than BIP and the generic MLP mix2 14.6% fewer. The ranking of the versions is **not stable**: mix1b is the best version at 2 MB and the worst at 8 MB (−11.1% for the generic MLP), and mix2 is the best at 8 MB. A second round of training did not give a reliable gain. We therefore do not claim a best version. What the result supports is only that training on the shared stream can turn a bad shared-cache policy into one that beats the best simple policy in this mixture, and that the ownership rule is not the way to get there. The experiment uses one test mixture and one seed, and the gains at 8 MB are on a case where BIP is already very good.

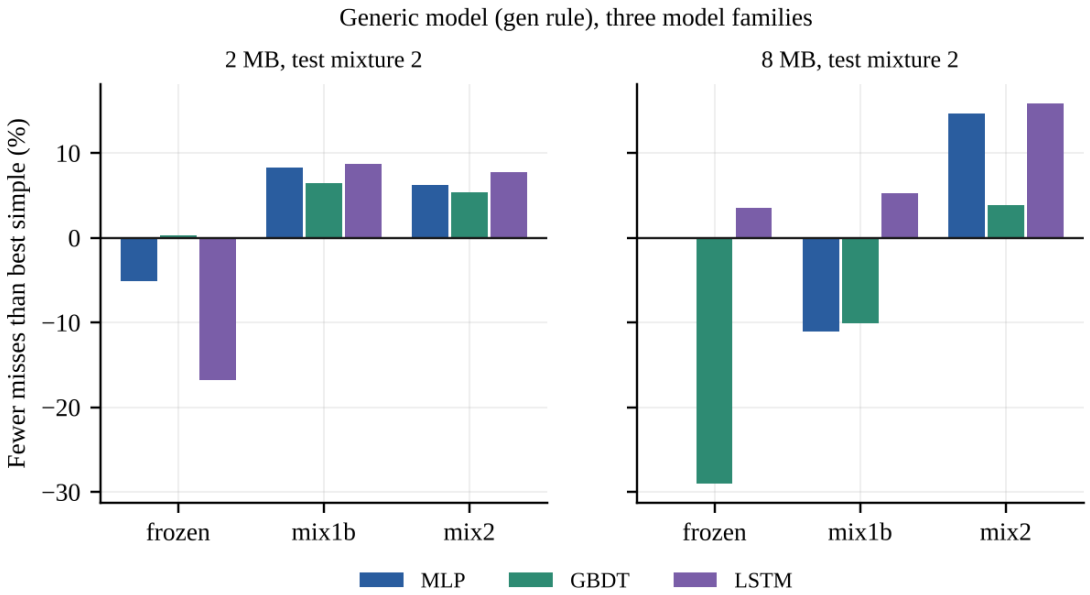


Figure 4.10: Extension 2: model families under the generic rule.

4.3.10 Static web simulation

The results can be explored in a static web page (`web_simulation/index.html`) that works offline from a folder. It has six screens: Overview, Single application, Switching, Shared cache, Decision inspector and Results and tables. Numbers that come from the complete runs carry a **Full run** badge. Animated replays carry a **Sample window** badge: they show 4,000 consecutive LLC accesses from the middle of one held-out test trace, with the cache state under each policy and the decision of the learned policy at each eviction. A window is a short stretch to look at, and not a statistic. The page does not run models in the browser; the replays were recorded by the C++ simulator and the frozen models.

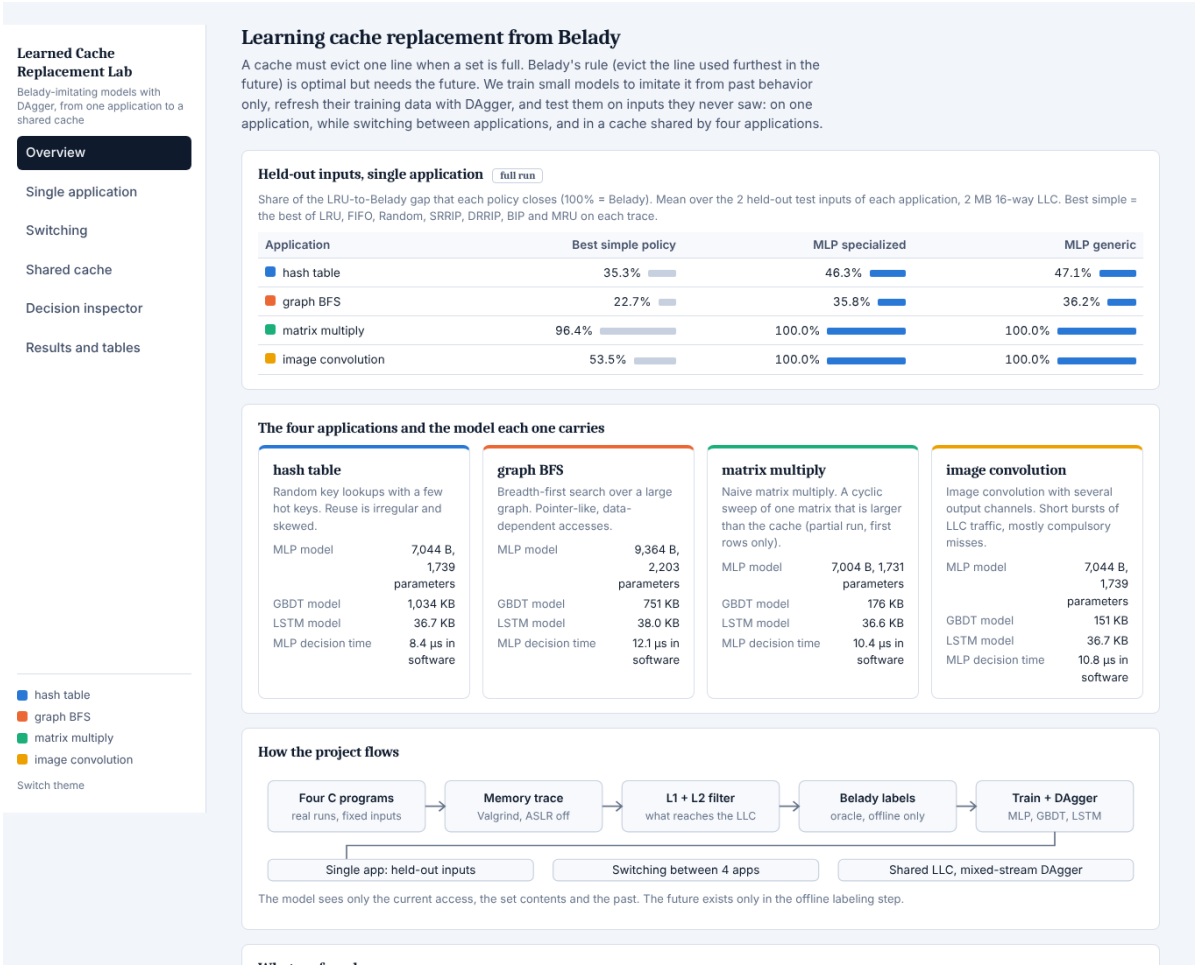


Figure 4.11: Web simulation: Overview screen (full-run results).

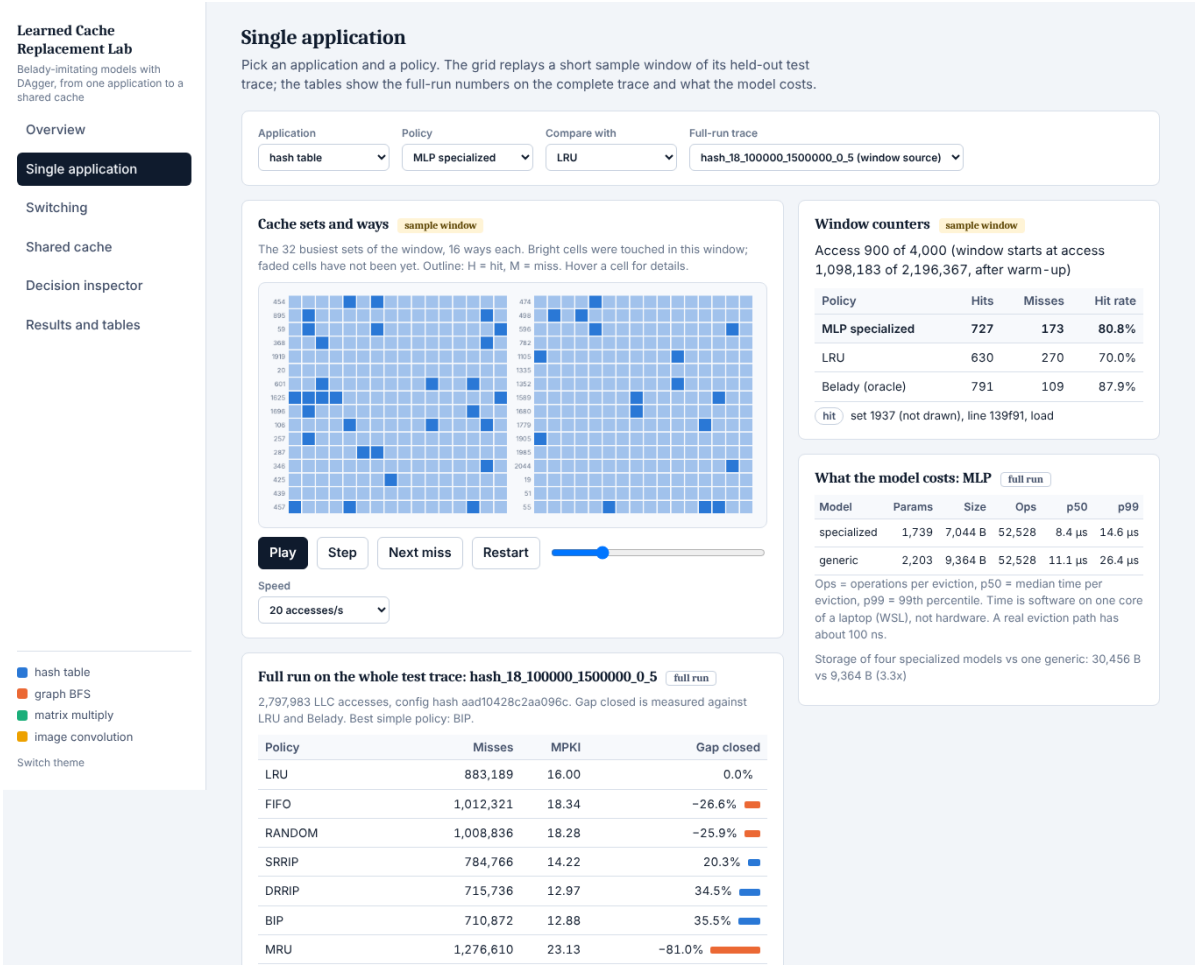


Figure 4.12: Web simulation: Single application screen with an animated sample window (hash, MLP specialized against LRU and Belady) and the full-run table.

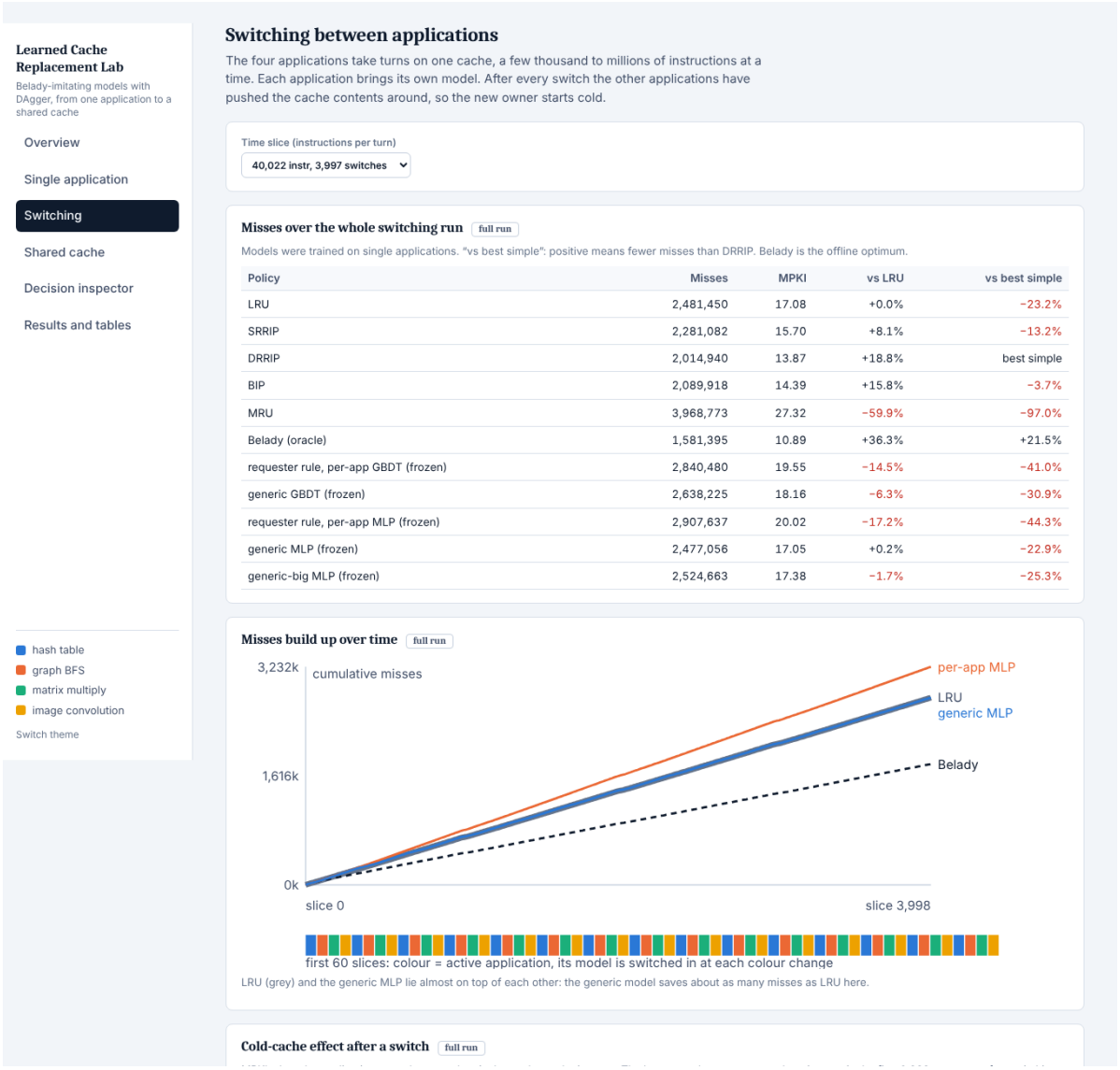


Figure 4.13: Web simulation: Switching screen (E7).

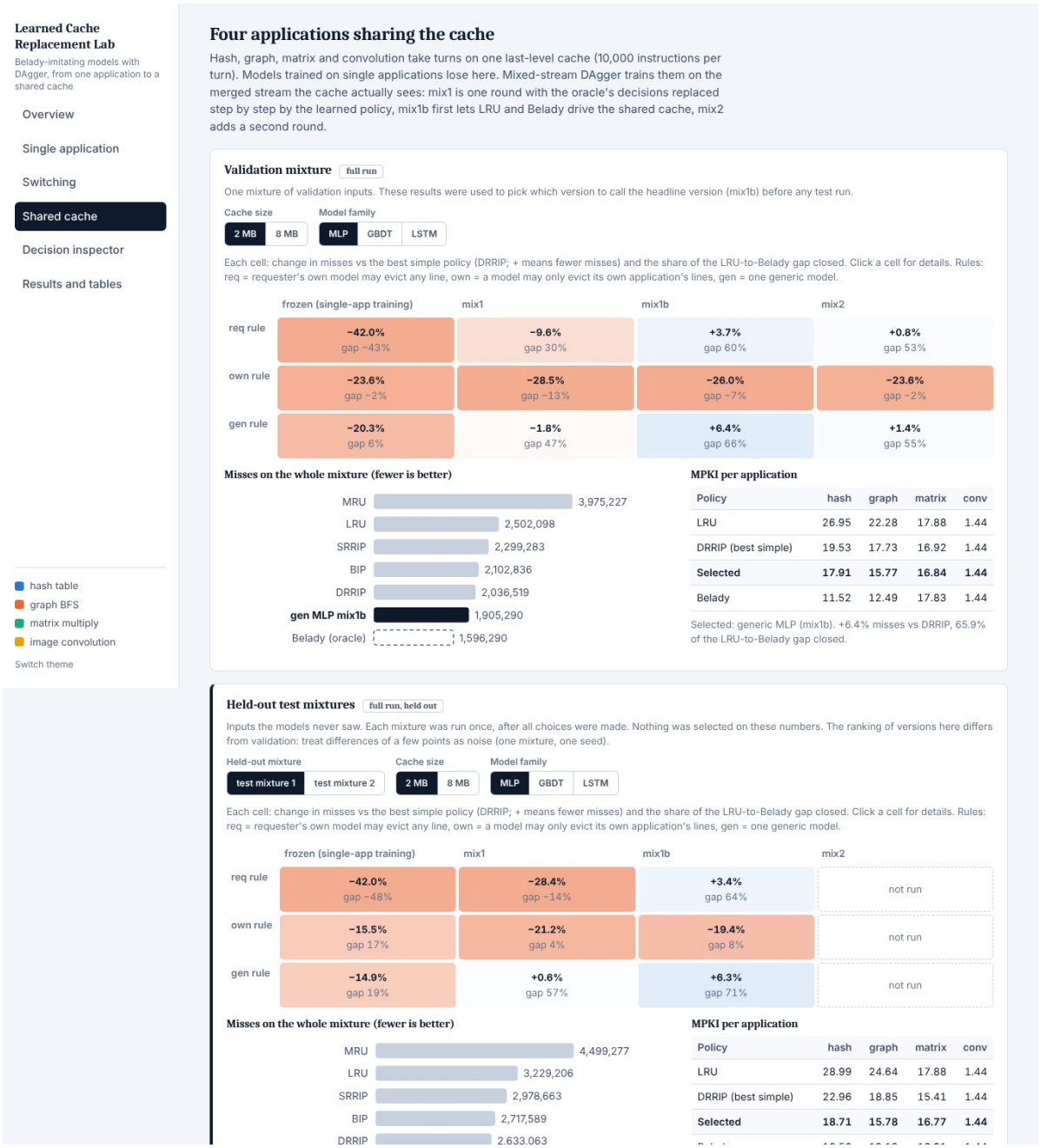


Figure 4.14: Web simulation: Shared cache screen, validation and held-out test results of the mixed-stream extension.

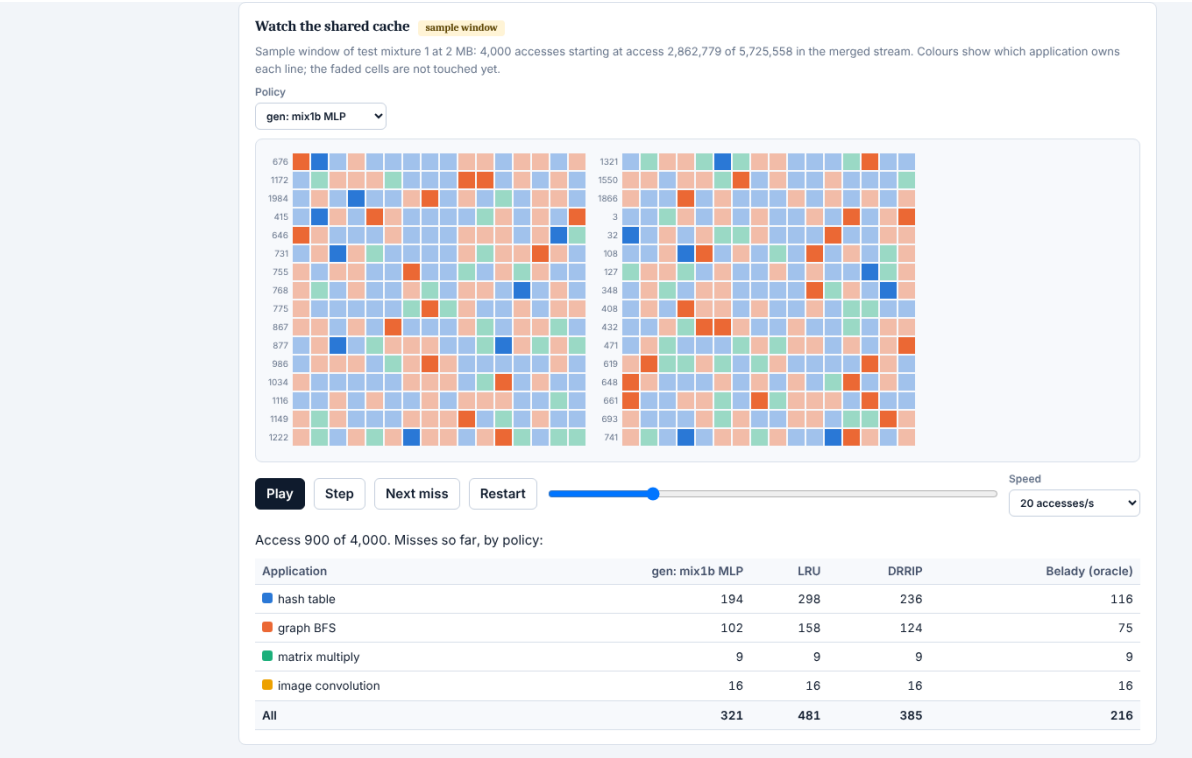


Figure 4.15: Web simulation: sample window of the shared cache (4,000 accesses), colours show the owning application.

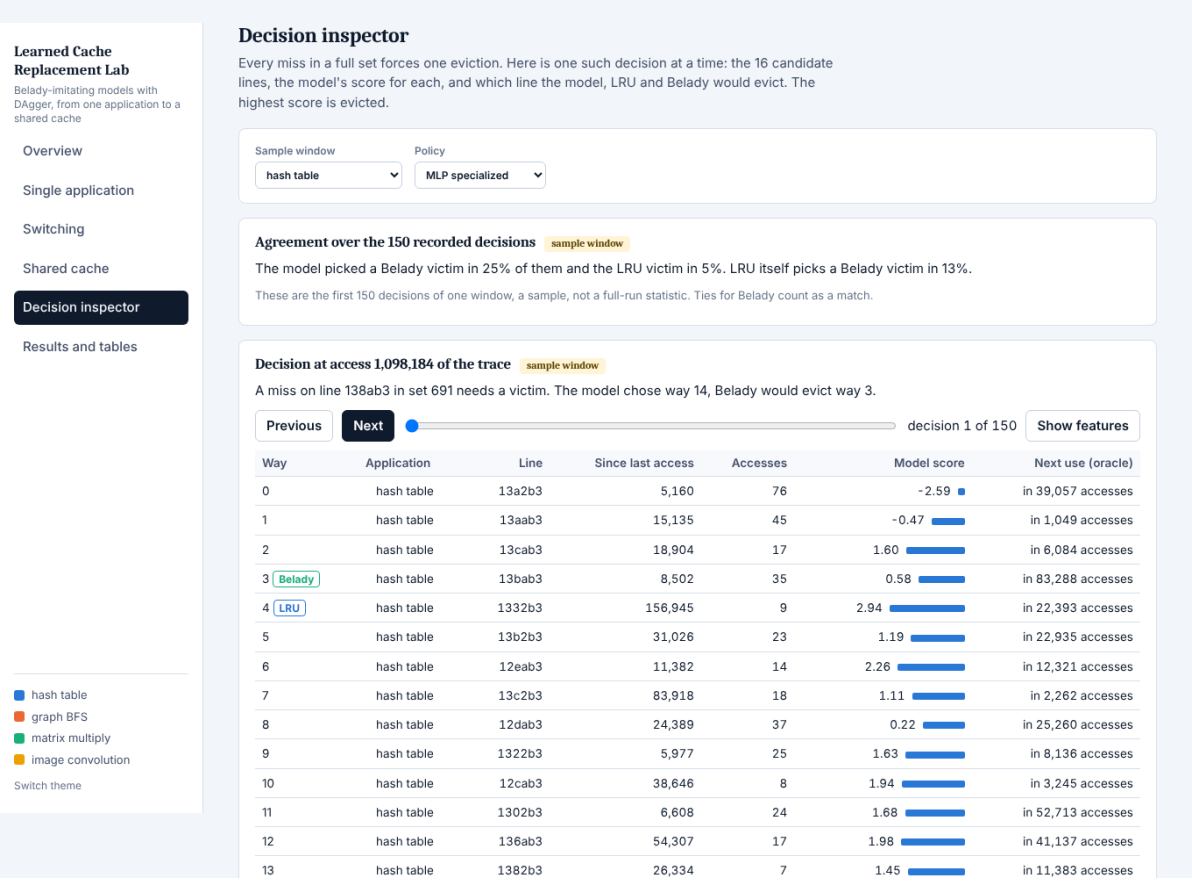


Figure 4.16: Web simulation: Decision inspector, one eviction with the 16 candidate lines, the model scores and the Belady and LRU victims.

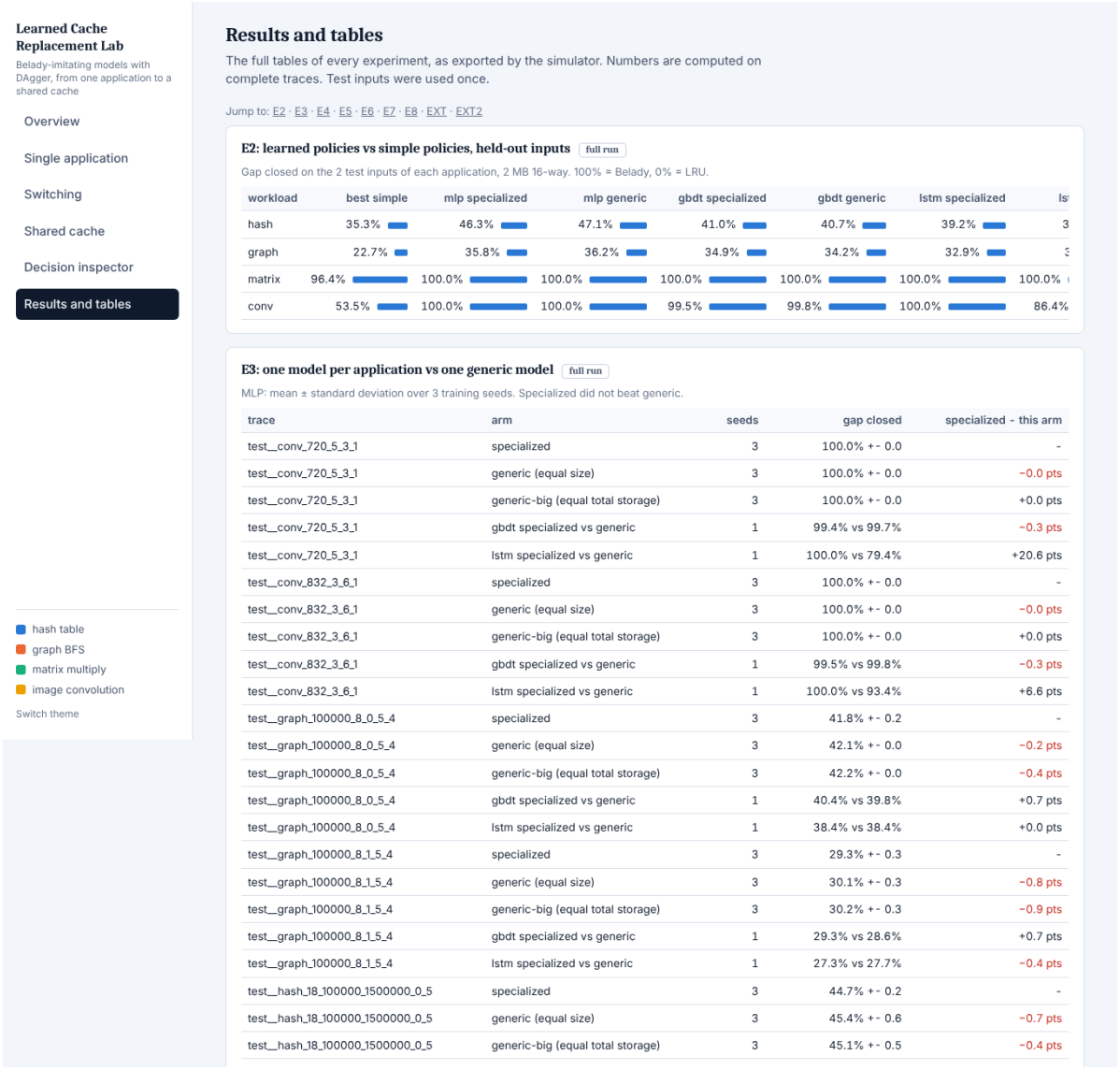


Figure 4.17: Web simulation: Results and tables screen.

4.4 Quality Assurance

Quality was ensured by the following measures. We list what they protect against and what they do not.

- **Test-before-use.** No component entered an experiment before its tests passed (Table 11).
- **Data discipline.** Training uses train inputs, all choices use the validation input, and each test input is run once per frozen model set. The main model set is stored with SHA-256 checksums in `model_set.json`; a marker file blocks a second test run. A test checks that the mixed-stream pipeline never reads the test split.
- **Reproducibility.** Fixed inputs and seeds, ASLR off, empty environment, a hash of the configuration in every result, a project log that records every decision.
- **Numbers from files.** Tables and figures in this report were generated from the result files by scripts; the web simulation reads the same files.
- **Honest comparison.** The best simple policy is chosen per trace and reported next to every learned result.

Limits. (1) The programs are four small, synthetic C workloads, not industry benchmarks such as SPEC. (2) There is no timing model and no prefetcher, so miss reductions are not speed-ups. (3) Software decision times are not hardware times. (4) Results depend on the 2 MB 16-way LLC and on the inputs; E5 shows only three other configurations. (5) The mixed-stream extension is one mixture and one seed per cell. (6) Model families were tuned only lightly. (7) We did not run the code of the related work, so no comparison with published numbers is claimed.

Chapter 5
Standards Adopted

5.1 Design Standards

The project follows no formal certification. We list the standards and conventions that we did follow and say how far.

- **Modular design.** Every stage is a separate program with a documented input and output (trace file, dataset file, model file, result file). File formats are written down in the repository (`docs/`).
- **Architecture description.** The system is described by the block diagram of Figure 1.1, the data-set naming of Table 9 and the arbitration rules of Figure 3.2, in the style of a conventional architecture description for a research simulator [13], [14].
- **Evaluation design.** We follow the usual machine-learning rule of separate training, validation and test data, and the architecture-research convention to compare with an offline optimal (Belady) and with the standard policies [2], [3], [13].
- **Fairness measure.** The Jain fairness index [17] is used for the shared cache, with the definition given in the original report.

5.2 Coding Standards

- **C++17** [18] for the simulator, labeler, multi-application engine and exporter. We compile with warnings on, and the code uses plain structs and header files in a small number of modules.
- **C** for the four workloads, to keep the memory behavior under control and the traces small.
- **Python** for training, experiment drivers and tests. We did not run a formal style checker on the whole code base, so we do not claim full compliance with PEP 8.
- **Shell scripts** for pipelines; they are resumable, print their configuration hash and fail loudly.
- **Naming and documentation.** Policy names, file tags and result columns are fixed words (for example `req`, `own`, `gen`, `dag1`, `mix1b`), and each decision about assumptions is recorded in the project log and in `docs/decision_log.md`.

5.3 Testing Standards

Test terminology (test case, test condition, expected result) follows ISO/IEC/IEEE 29119-1 [15]. We used unit tests (policy hand checks, labeler, parser), property tests against an exhaustive search (Belady), parity tests (C++ against Python decisions), mutation tests (a removed PC mask and a changed feature constant had to make the tests fail), integration tests (end-to-end pipeline on small synthetic traces) and leakage tests (no test split in training). Table 11 lists them.

Cost analysis

The project needs no purchased hardware or software. All tools are open source (a C++ compiler, Valgrind, Python, PyTorch, LightGBM, docx and matplotlib for this report) and everything ran on one personal laptop. The main cost was time: the long jobs (labeling, DAgger rounds, the experiment runs) took many hours, and several had to be restarted after the laptop went to sleep, which is why the scripts are resumable. The results take about 27 GB of disk space; the code, configurations, models and small results fit in a few tens of megabytes.

The cost of the **proposed design**, if it were built, is a different question and was not measured. Table 16 gives the storage and software time that each model needs. They show that a hardware version would need a much smaller and cheaper model, for example a small table lookup, and we do not claim that the models in this report fit such a budget.

Chapter 6
Conclusion and Future Scope

6.1 Conclusion

We built a complete simulation pipeline to study whether small models that imitate Belady's algorithm can improve last-level cache replacement, and whether such models should be carried by each application. The answers to our four research questions are as follows.

1. **RQ1: Yes, on held-out inputs of the same application.** The MLP closes 46% to 47% (hash) and 36% (graph) of the LRU-to-Belady gap, against 35% and 23% for the best simple policy, and 100% on matrix and conv. DAgger round 1 was the largest single improvement; round 2 gave no consistent gain.
2. **RQ2: No advantage for specialization.** One generic model of the same size was as good as, or slightly better than, the specialized models and needs 3.3 to 4.6 times less storage than four specialized ones. Models trained on one application fail on others (as low as −425% gap closed).
3. **RQ3: The software cost is high.** 7 KB to 1 MB per model and 7 to 146 µs per decision, against a hardware budget of about 100 ns. Under our cost model, activating per-application models at a context switch never paid off; the frozen per-application models lost to the best simple policy at every slice length.
4. **RQ4: Single-application models fail in a shared cache; shared-stream DAgger fixes much of it.** Under E8 the generic MLP stayed below DRRIP. With mixed-stream DAgger, generic models beat the best simple policy by 5% to 9% at 2 MB and up to 16% at 8 MB on a second, unseen mixture. The ownership rule did not help. Which DAgger version is best was not stable.

In short, a learned policy can beat the best simple policy in simulation, both for single applications and for a shared cache, if it is trained on the states it creates itself. The evidence for application-carried models, however, is negative: a generic model does as well, and the cost of switching models was not paid back. The practical value of the work is the demonstrated method, the evaluation discipline and the negative results, not a ready design.

6.2 Future Scope

- **More and larger workloads.** Standard benchmark traces (for example SPEC or GAP) and more than one trace per application would test whether the gains hold on real programs.
- **Timing and prefetching.** Add a timing model, a prefetcher and write-back traffic to turn miss reductions into speed-up estimates.
- **Hardware-sized models.** Distill the MLP into a tiny table or a small tree that fits 100 ns, and compare with Hawkeye- and Mockingjay-style predictors [4], [7]. This is the most important open step.
- **Real shared caches.** Several cores, application-aware partitioning, more than four applications, and longer DAgger with more seeds and mixtures to test whether the ranking of the versions becomes stable.
- **Cost model.** Measure the activation cost on a real operating system or in a full-system simulator instead of using the assumed parameters.
- **Online learning.** Let the policy adapt while the application runs, which would remove the need to carry a trained model.

- **Live in-browser mode.** The web simulation only replays recorded windows. A live mode would run a simulator and the MLP in the browser on fresh traces. It needs a JavaScript port of the policies and of the feature code, tested decision by decision against the C++ simulator, and traces that match the training data.

References

[1] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[2] A. Jaleel, K. B. Theobald, S. C. Steely Jr. and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," in *Proc. 37th Int. Symp. on Computer Architecture (ISCA)*, 2010, pp. 60–71.

[3] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely Jr. and J. Emer, "Adaptive insertion policies for high performance caching," in *Proc. 34th ISCA*, 2007, pp. 381–391.

[4] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. 43rd ISCA*, 2016, pp. 78–89.

[5] Z. Shi, X. Huang, A. Jain and C. Lin, "Applying deep learning to the cache replacement problem," in *Proc. 52nd IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, 2019, pp. 413–425.

[6] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan and J. Ahn, "An imitation learning approach for cache replacement," in *Proc. 37th Int. Conf. on Machine Learning (ICML)*, PMLR 119, 2020.

[7] I. Shah, A. Jain and C. Lin, "Effective mimicry of Belady's MIN policy," in *Proc. IEEE Int. Symp. on High-Performance Computer Architecture (HPCA)*, 2022.

[8] S. Ross, G. Gordon and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, PMLR 15, 2011, pp. 627–635.

[9] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017, pp. 3146–3154.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2007, pp. 89–100.

[12] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[13] A. Jain and C. Lin, *Cache Replacement Policies*. Morgan & Claypool, 2019.

[14] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[15] *ISO/IEC/IEEE 29119-1: Software and systems engineering — Software testing — Part 1: General concepts*, 2022.

[16] D. Bahdanau, K. Cho and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. on Learning Representations (ICLR)*, 2015.

[17] R. Jain, D.-M. Chiu and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corp., Tech. Rep. DEC-TR-301, 1984.

[18] *ISO/IEC 14882:2017, Programming languages — C++*, 2017.

INDIVIDUAL CONTRIBUTION REPORT

LEARNING CACHE REPLACEMENT FROM BELADY: DAGGER TRAINING FROM SINGLE APPLICATIONS TO SHARED CACHES

SHIVANSH BATRA
23051784

Abstract: This project studies whether small machine-learning models can replace the heuristic that chooses which line to evict from the last-level cache. A simulator, a Belady labeler, three model families and DAgger training were built and tested on four real programs, and the models were also tested when four programs share the cache. Learned policies beat the best simple policy in simulation, but one generic model is as good as per-application models, and software decision times are far above a hardware budget.

Individual contribution and findings: Responsible for section 4 of the project, the shared last-level cache. This covers the multi-application engine in which the four programs take turns on one cache in rounds of 10,000 instructions, the owner bits that keep the lines of different programs apart, and the three arbitration rules req, own and gen. It also covers the switching experiment E7, the shared-cache experiment E8 and the mixed-stream DAgger extension. Findings of this section: models trained on one program lose to the best simple policy when four programs share the cache; generic models trained on the shared stream have 5% to 9% fewer misses than DRRIP at 2 MB and up to 16% fewer than BIP at 8 MB on a held-out mixture; the ownership rule never beat the best simple policy.

Individual contribution to project report preparation: Section 2.7, the arbitration rules in Section 3.3.2, and Sections 4.3.6 to 4.3.9 with their tables and figures.

Individual contribution for project presentation and demonstration: Presents the shared-cache part and demonstrates the Switching and Shared cache screens of the web simulation.

Full Signature of Supervisor: Full signature of the student:

....

INDIVIDUAL CONTRIBUTION REPORT

LEARNING CACHE REPLACEMENT FROM BELADY: DAGGER TRAINING FROM SINGLE APPLICATIONS TO SHARED CACHES

PRIYAM KUMAR
23051690

Abstract: This project studies whether small machine-learning models can replace the heuristic that chooses which line to evict from the last-level cache. A simulator, a Belady labeler, three model families and DAgger training were built and tested on four real programs, and the models were also tested when four programs share the cache. Learned policies beat the best simple policy in simulation, but one generic model is as good as per-application models, and software decision times are far above a hardware budget.

Individual contribution and findings: Responsible for section 1 of the project, the workloads, traces and dataset. This covers the four C programs (hash table, graph traversal, matrix multiplication, convolution) with fixed training, validation and test inputs, the Valgrind traces of the region of interest, the L1 and L2 filter that produces the last-level cache trace, and the Belady labeler that writes one training row per eviction. Findings of this section: each row holds 16 candidate lines with past-only features and the set of Belady victims as label; the next use of a line is label information only and never reaches a model.

Individual contribution to project report preparation: Sections 4.1.1 to 4.1.3, Tables 6 and 7 and the data-flow description around Figure 1.1.

Individual contribution for project presentation and demonstration: Presents the path from program to dataset and shows a dataset row with its fields.

Full Signature of Supervisor: Full signature of the student:

....

INDIVIDUAL CONTRIBUTION REPORT

LEARNING CACHE REPLACEMENT FROM BELADY: DAGGER TRAINING FROM SINGLE APPLICATIONS TO SHARED CACHES

SHRISTI MISHRA

23051788

Abstract: This project studies whether small machine-learning models can replace the heuristic that chooses which line to evict from the last-level cache. A simulator, a Belady labeler, three model families and DAgger training were built and tested on four real programs, and the models were also tested when four programs share the cache. Learned policies beat the best simple policy in simulation, but one generic model is as good as per-application models, and software decision times are far above a hardware budget.

Individual contribution and findings: Responsible for section 3 of the project, the models and single-application DAgger. This covers the MLP, LightGBM and LSTM models, specialized and generic versions, DAgger rounds 1 and 2, and experiments E2 to E5. Findings of this section: the MLP closes 46% to 47% of the LRU-to-Belady gap on hash and 36% on graph against 35% and 23% for the best simple policy; one generic model is as good as the four specialized ones and needs 3.3 to 4.6 times less storage; models do not transfer between applications; DAgger round 1 gave the largest gain and round 2 no consistent one.

Individual contribution to project report preparation: Sections 2.5 and 2.6, Sections 4.1.4 and 4.1.5, and Sections 4.3.2 to 4.3.4 with Tables 12 to 15.

Individual contribution for project presentation and demonstration: Presents the models and DAgger with a worked example and discusses the results of E2 to E5.

Full Signature of Supervisor: Full signature of the student:

....

INDIVIDUAL CONTRIBUTION REPORT

LEARNING CACHE REPLACEMENT FROM BELADY: DAGGER TRAINING FROM SINGLE APPLICATIONS TO SHARED CACHES

JATIN ARYAN
23053366

Abstract: This project studies whether small machine-learning models can replace the heuristic that chooses which line to evict from the last-level cache. A simulator, a Belady labeler, three model families and DAgger training were built and tested on four real programs, and the models were also tested when four programs share the cache. Learned policies beat the best simple policy in simulation, but one generic model is as good as per-application models, and software decision times are far above a hardware budget.

Individual contribution and findings: Responsible for section 2 of the project, the cache simulator, baseline policies and Belady. This covers the set-associative cache simulator with L1, L2 and last-level cache, the policies LRU, FIFO, Random, MRU, SRRIP, BRRIP, DRRIP and BIP, Belady as the offline optimum, the gap-closed metric, the test programs that check these components, and experiment E1. Findings of this section: the simulator reproduces hand-computed results and Belady matches an exhaustive search on small traces; on hash and graph the best simple policy closes only 35% and 23% of the LRU-to-Belady gap, so a large gap remains.

Individual contribution to project report preparation: Sections 2.1 to 2.4, Section 4.2 with Table 11, and Section 4.3.1.

Individual contribution for project presentation and demonstration: Explains how the cache simulation works with a small example and shows the baseline policies and Belady.

Full Signature of Supervisor: Full signature of the student:

....

INDIVIDUAL CONTRIBUTION REPORT

LEARNING CACHE REPLACEMENT FROM BELADY: DAGGER TRAINING FROM SINGLE APPLICATIONS TO SHARED CACHES

SHREYA SINGH

23051954

Abstract: This project studies whether small machine-learning models can replace the heuristic that chooses which line to evict from the last-level cache. A simulator, a Belady labeler, three model families and DAgger training were built and tested on four real programs, and the models were also tested when four programs share the cache. Learned policies beat the best simple policy in simulation, but one generic model is as good as per-application models, and software decision times are far above a hardware budget.

Individual contribution and findings: Responsible for section 5 of the project, the evaluation, the web simulation and the report. This covers experiment E6 on model size and software decision time (the MLP needs 7 to 9 KB and 8 to 12 microseconds per decision, far above a hardware budget of about 100 nanoseconds), the result tables and figures that are generated from the result files, the cost model, the static web simulation with its full-run results and sample windows of 4,000 accesses, and the checking of numbers in the report against the result files. Findings of this section: the web simulation shows the same recorded results on every run, and its numbers match the report tables.

Individual contribution to project report preparation: Chapters 1, 3, 5 and 6, the abstract, and the checking and assembly of the whole report.

Individual contribution for project presentation and demonstration: Demonstrates the web simulation and presents the evaluation and conclusions.

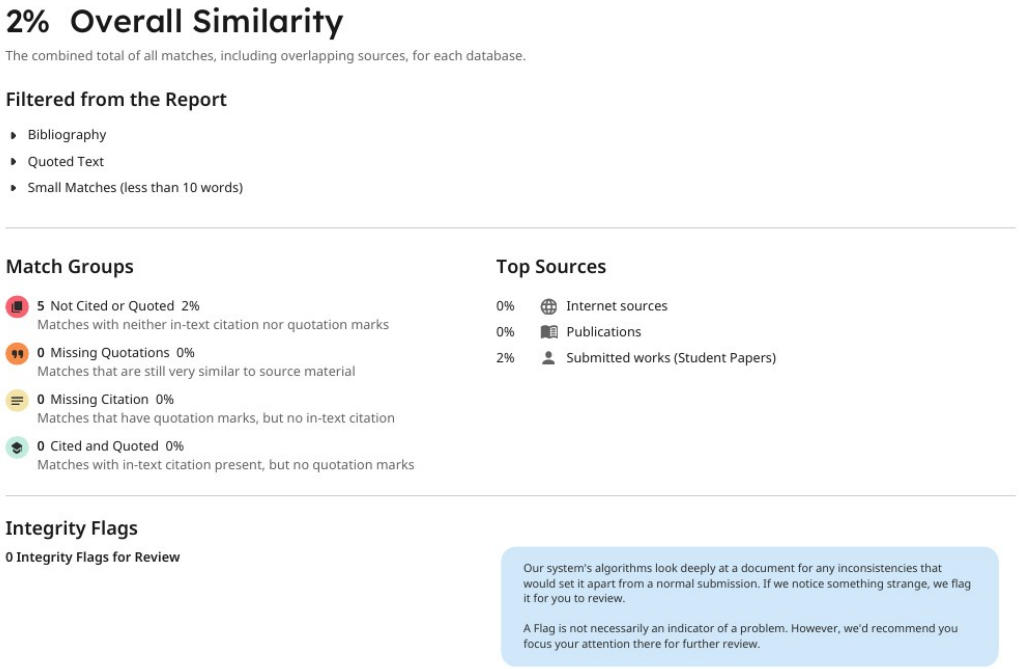
Full Signature of Supervisor: Full signature of the student:

....

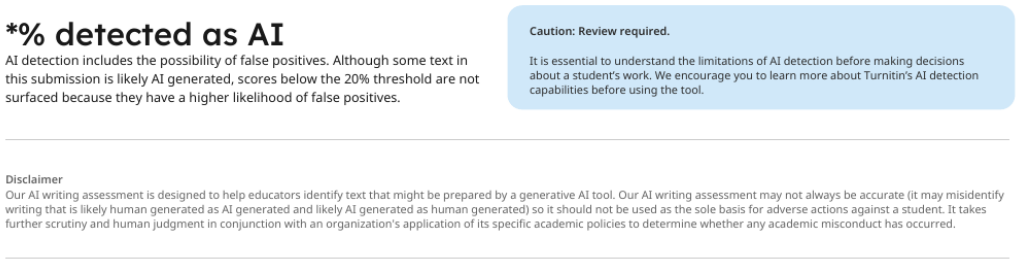
TURNITIN PLAGIARISM REPORT

(This report is mandatory for all the projects and plagiarism must be below 25%)

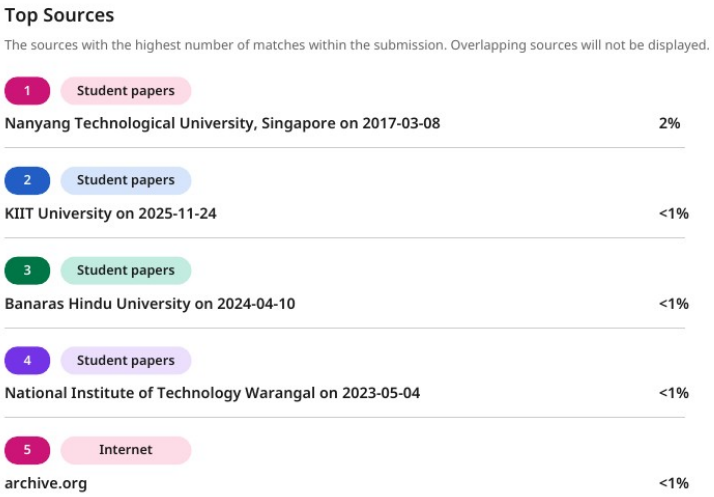
Similarity index: 2 %. Bibliography, quoted text and matches of fewer than 10 words are excluded by Turnitin; the report shows 0 integrity flags. **AI writing indicator:** shown by Turnitin as "*%", its mark for a score below its 20 % display threshold. Submission ID trn:oid:::3618:157912315, submitted on 10 November 2026; Turnitin counted 35 pages and 9,871 words in the submitted file.



Turnitin Integrity report: overview (similarity 2 %, 0 integrity flags).



Turnitin AI writing report: overview.



Turnitin Integrity report: top sources. The highlighted matches are page footers, standard section headings and short common phrases.